



PROJET DE FIN D'ETUDES
5ème Année en Ingénierie Informatique et Réseaux

Conception et développement de la SANDBOX ConnectAPI

Réalisé par :

ROUDALI Walid

Tuteur (s) :

Encadrant Professionnel : Mr. ZABIR Adil

Encadrant Pédagogique : Mr. YOUSSEFI Mohamed.

Au sein de (Organisme d'accueil) :



Dédicace

“

Je dédie ce rapport à toutes les personnes qui m'ont soutenu et encouragé tout au long de ce projet.

À ma petite famille, ma mère, mon père, ma sœur, mon beau-frère et ma nièce Ferdouass, pour leur amour inconditionnel, leur soutien constant et leur compréhension. Votre présence et votre soutien ont été ma source de force et de motivation.

À mes amis, pour leur soutien moral, leurs encouragements et leurs moments de détente qui m'ont permis de garder un équilibre sain pendant cette période intense.

À mes enseignants, pour leur guidance, leur expertise et leur engagement à me pousser vers l'excellence. Vos conseils et votre soutien ont été inestimables dans ma croissance personnelle et professionnelle.

Enfin, je tiens à exprimer ma gratitude envers moi-même pour ma persévérance, ma détermination et ma volonté de relever les défis tout au long de ce projet. C'est grâce à mon engagement et à ma passion que nous avons pu atteindre nos objectifs.

Merci.

”

Remerciement

Tout d'abord, je remercie Allah le tout puissant de m'avoir donné le courage et la patience nécessaires à mener ce travail à son terme.

Je tiens à remercier tout particulièrement mon encadrant au sein de la société HPS M. **Adil ZABIR** Project Manager, qui m'a accordé son soutien et sa confiance tout au long de cette expérience professionnelle. Ses précieux conseils, son encadrement et sa disponibilité ont grandement enrichi mon apprentissage.

Je désire remercier également M. **Rachid BAHADI**, M. **Reda LAHRACHE** et M. **Ayoub ERRABI** des ingénieurs chez HPS, pour les renseignements précieux qu'ils m'ont fournis, ainsi que pour leurs disponibilités et leurs orientations. Je remercie également toute l'équipe de la société HPS qui ont toujours été à l'écoute de mes moindres besoins.

J'adresse mes vifs remerciements à Pr. **YOUSSEFI Mohamed**, mon encadrant de l'Ecole Marocaine des Sciences de l'Ingénieur pour son aide immense, la qualité de son suivi ainsi que pour tous les conseils et les informations qu'il m'a prodigués avec un degré de patience et de professionnalisme sans égal.

Que les membres de jury trouvent, ici, l'expression de mes sincères remerciements pour l'honneur qu'ils me font en prenant le temps de lire et d'évaluer ce travail.

Merci à toutes ces personnes qui ont contribué de près ou de loin à la réussite de ce projet. Votre soutien et votre implication ont été d'une valeur inestimable, et je vous en suis profondément reconnaissant.

Résumé

Le présent projet synthétise le travail réalisé dans le cadre de notre projet de fin d'études qui vient de conclure notre formation d'ingénieur d'état en Ingénierie Informatique et Réseaux à l'EMSI. Le projet a été réalisé chez HPS (Hightech Payment Systems) à Casablanca.

L'objectif de notre projet consiste à optimiser la Sandbox en repensant sa fonctionnalité et son interface d'administration. Dans ce rapport de stage, nous présentons les différentes étapes réalisées pour la conception et le développement de la solution apportée.

Dans ce sens, nous avons situé le projet dans son cadre pour une meilleur compréhension du sujet. Puis, nous avons fait une étude fonctionnelle et technique. Ensuite, nous avons fait l'analyse et la conception du Sandbox ConnectAPI. Enfin, nous avons présenté le résultat du travail que nous avons fait.

Abstract

This report summarizes the work carried out as part of our end-of-study project as engineering student at EMSI to obtain the engineering degree in Computer Engineering and Networks. This document constitutes a summarization of our graduation project at HPS (Hightech Payment Systems) in Casablanca.

The objective of our project is to optimize the Sandbox by redesigning its functionality and its administration interface. In this document, we present the various steps taken for the design and development of the proposed solution.

In this regard, we have placed the project in its context for a better understanding of the subject. Then, we conducted a functional and technical study. Next, we analyzed and designed the Sandbox ConnectAPi. Finally, we presented the results of the work we have done.

Liste des abréviations

Abréviation	Désignation
HPS	Hightech Payment Systems
API	Application Programming Interface
YAML	Yet Another Markup Language
UML	Unified Modeling language
JAKRTAA	Java Enterprise Edition
QQOQCP	Qui-Quoi-Ou-Quand-Comment-Pourquoi
UI	Interface utilisateur
REST	REpresentational State Transfer
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
CSS	Cascading Style Sheets
DOM	Document Object Model
SPA	Single Page Application
SOA	Architecture orientée Service
NPM	Node Package Manager

Liste des figures

Figure 1 - Logo HPS	15
Figure 2 - Actionnariat de HPS dans les entreprises collaboratrices	15
Figure 3 - Organigramme de HPS.....	16
Figure 4 - Les activités de HPS.....	17
Figure 5 - Architecture de PowerCARD	17
Figure 6 - Vue globale de la méthode SCRUM.	20
Figure 7 - Sprints par fichier Excel.....	21
Figure 8 - Diagramme de Gantt.....	22
Figure 9 - Uml logo	29
Figure 10 - Diagramme de Cas d'Utilisation global de la Sandbox ConnectApi HPS.....	30
Figure 11 - Diagramme de Séquence du cas de Test des APIs par l'utilisateur	32
Figure 12 - Diagramme de Séquence pour l'Administrateur : Cas d'Activation d'un Compte Utilisateur	33
Figure 13 - Diagramme de class	34
Figure 14 - Architecture technique du système.....	37
Figure 15 - Jakarta EE logo	39
Figure 16 - Spring Boot logo.....	39
Figure 17 - logo Spring data JPA.....	40
Figure 18 - JAVASCRIPT logo	40
Figure 19 - REACTJS logo.....	41
Figure 20 - Tailwind CSS Logo	41
Figure 21 - Gradle Logo	41
Figure 22 - Npm Logo	42
Figure 23 - JUnit 5	42
Figure 24 - Visual studio code.....	42
Figure 25 - Postman Logo	43
Figure 26 - IntelliJ Idea logo	43
Figure 27 - flyway Logo	44
Figure 28 - Bitbucket Logo	44
Figure 29 - Git Logo	45
Figure 30 - OpenApi Logo.....	45
Figure 31 - Swagger Logo	46
Figure 32 - StopLight LOGO.....	46
Figure 33 - L'interface Accueil sandbox-ui	48
Figure 34 - l'interface Login sandbox-ui.....	49
Figure 35 - L'interface inscription sandbox-ui	49
Figure 36 - L'interface Produits light mode Sandbox-ui.....	50
Figure 37 - L'interface product release sandbox-ui	51
Figure 38 - L'interface Solution sandbox-ui	51
Figure 39 - L'interface module sandbox-ui	52
Figure 40 - L'interface Api sandbox-ui	52
Figure 41 - L'interface produit sandbox-admin	53
Figure 42 - l'interface product release sandbox-admin.....	54
Figure 43 – L'interface solution sandbox-admin.....	54

Figure 44 - l'interface module sandbox-admin	55
Figure 45 - L'interface utilisateur sandbox-admin	55
Figure 46 - L'interface d'ajout d'un utilisateur sandbox-admin	56
Figure 47 - L'interface de modification d'un utilisateur sandbox-admin.....	56
Figure 48 - L'interface de suppression d'un utilisateur sandbox-admin.....	57

Liste des tableaux

Tableau 1 - La méthode QQQQCP	25
-------------------------------------	----

Table des matières

Dédicace	2
Remerciement.....	3
Résumé.....	4
Abstract.....	5
Liste des abréviations	6
Liste des figures.....	7
Liste des tableaux.....	9
Table des matières.....	10
Introduction générale	13
Chapitre 1	14
1 Contexte générale du projet.....	15
1.1 Présentation de l'organisme d'accueil	15
1.1.1 L'entreprise HPS « Hightech Payment Systems »	15
1.1.2 Historique de HPS.....	16
1.1.3 Organigramme de l'entreprise	16
1.1.4 Les activités de HPS	17
1.1.5 La solution monétique PowerCard.....	17
1.2 Cadre du projet.....	18
1.2.1 Présentation du projet	18
1.2.2 Objectif du Stage	18
1.2.3 Missions et tâches	19
1.3 Dossier de pilotage	19
1.3.1 Environnement et Méthodologie de travail.....	19
1.3.2 La méthode Agile.....	19
1.3.3 La méthode Scrum	20
1.3.4 Pourquoi Scrum ?	20
1.3.5 Planification du projet.....	21
Conclusion.....	22
Chapitre 2	23
2 Spécification et analyse.....	24
2.1 Etudes de l'existant	24
2.2 Problématique	24

2.3	Solution.....	24
2.4	Périmètre du projet.....	25
2.4.1	Analyse des besoins	26
	Conclusion.....	27
Chapitre 3		28
3	Étude conceptuelle	29
3.1	Formalisme UML.....	29
3.2	Diagramme de cas d'utilisation	30
3.2.1	Définition des acteurs.....	31
3.2.2	Description des cas d'utilisation	31
3.3	Diagramme de séquence.....	31
3.3.1	Diagramme de Séquence du cas de Test des APIs par l'utilisateur	32
3.3.2	Diagramme de Séquence pour l'Administrateur : Cas d'Activation d'un Compte	33
3.4	Diagramme de classe.....	34
	Conclusion.....	35
Chapitre 4		36
4	Architecture technique	37
4.1	Architecture logicielle du système	37
4.2	Environnement de Développement	38
4.2.1	Choix technologique.....	38
4.2.2	Langages et Frameworks.....	39
4.2.3	Outils de développement.....	42
	Conclusion.....	46
Chapitre 5		47
5	Mise en œuvre	48
5.1	Sandbox-UI	48
5.1.1	L'interface Accueil.....	48
5.1.2	L'interface Login	49
5.1.3	L'interface inscription	49
5.1.4	L'interface produits.....	50
5.1.5	L'interface product release.....	51
5.1.6	L'interface Solution	51
5.1.7	L'interface Module.....	52
5.1.8	L'interface API.....	52
5.2	Sandbox-Admin	53
5.2.1	L'interface produits	53

5.2.2	L'interface product release.....	53
5.2.3	L'interface Solution	54
5.2.4	L'interface Modules	54
5.2.5	Gestion des utilisateurs	55
	Conclusion.....	57
	Conclusion générale	58
	Webographie	59

Introduction générale

Les APIs (Interfaces de Programmation d'Applications) constituent des ensembles de définitions et de protocoles qui permettent aux applications logicielles de communiquer entre elles. Elles définissent les moyens d'accéder aux données et aux fonctionnalités d'une application de manière standardisée, favorisant ainsi l'intégration et l'échange d'informations entre des applications tierces. Dans le paysage actuel du développement d'applications modernes, les APIs jouent un rôle crucial en promouvant l'interopérabilité et la modularité des systèmes logiciels.

Les APIs révolutionnent le monde du développement logiciel en offrant une multitude d'avantages aux développeurs et aux utilisateurs finaux. Elles favorisent la réutilisation du code, permettant aux développeurs de se concentrer sur la création de nouvelles fonctionnalités plutôt que de réécrire du code existant. De plus, les APIs stimulent l'agilité et l'innovation en facilitant la collaboration entre les équipes de développement et la création de solutions innovantes en combinant des services existants.

Face à l'adoption croissante des APIs, le projet de fin d'études se concentre sur l'optimisation de la Sandbox ConnectAPI en repensant sa fonctionnalité et son interface d'administration. L'objectif central est de développer un espace administratif intuitif qui simplifie la gestion des documentations d'API et permet une personnalisation avancée des espaces clients. Ce projet vise à créer une interface graphique flexible et conviviale, permettant aux administrateurs de gérer efficacement les APIs et d'adapter l'interface aux besoins spécifiques de chaque client.

Ce rapport de fin d'études se structure en cinq chapitres pour vous présenter le projet dans son intégralité. Le premier chapitre vous plongera dans le contexte général, en exposant le cadre du sujet, le dossier de pilotage et les objectifs du projet. Ensuite, le chapitre de Spécifications et Analyse se divise en deux parties. La première analysera l'environnement actuel et ses limitations. La seconde reformulera la problématique à résoudre et définira précisément le périmètre du projet. Le rapport se poursuit avec l'étude conceptuelle qui présente les concepts clés et les solutions envisagées pour répondre à la problématique. L'architecture technique détaille ensuite la structure logicielle et matérielle du projet, expliquant les technologies employées. Le chapitre de mise en œuvre vous plonge au cœur du développement. Enfin, la conclusion résume les points clés et les réalisations du projet, l'évaluant par rapport aux objectifs initiaux et proposant des perspectives d'évolution futures.

Chapitre 1

Contexte général du projet

Ce chapitre vise à situer le projet dans son contexte organisationnel et opérationnel. Dans un premier temps, il présente l'organisme d'accueil, HPS. Ensuite, il décrit le cadre général du projet. Enfin, la dernière partie est consacrée à l'approche adoptée pour mener le projet à bien.

1 Contexte générale du projet

1.1 Présentation de l'organisme d'accueil

1.1.1 L'entreprise HPS « Hightech Payment Systems »

HPS est une société marocaine, multinationale spécialisée dans le développement de solutions et services de paiement électronique pour les institutions financières émetteurs, acquéreurs, processeurs, opérateurs de réseaux mobiles, la distribution, ainsi que les switches nationaux et régionaux dans le monde. HPS, acteur majeur de l'industrie du paiement électronique. Fondée en 1995, HPS est devenue un leader mondial dans son domaine, avec des solutions utilisées dans plus de 90 pays. Ses produits comprennent notamment PowerCARD, une plateforme de paiement électronique complète, et DMP, une solution de gestion de la monétique.



Figure 1 - Logo HPS

Les clients de HPS sont les Banques commerciales – Banques Centrales Institutions de microfinance Switchs nationaux et régionaux - Retailers Opérateurs Télécoms – Pétroliers. Les solutions de HPS englobent toute la chaîne de valeur du paiement et couvrent tant la gestion de l'émission, de l'acquisition, du switch que l'ensemble des activités de paiement sur une seule et unique plateforme. « PowerCard » est, en effet, une solution hautement paramétrable qui permet à ses utilisateurs une personnalisation et une réactivité maximales pour s'adapter à la nature de leurs activités et accompagner leur évolution. Doté d'une interface ergonomique et intuitive, PowerCard offre un grand confort d'utilisation et réduit sensiblement les coûts d'intégration, de développement et de formation à de nouvelles solutions liées au lancement de nouveaux produits. Certifiée PA-DASS et par l'ensemble des organismes de paiement internationaux tels que Visa, MasterCard, Diners Club, Japan Credit Bureau, American Express et China Union Pays, PowerCard permet une acceptation globale des réseaux de paiement internationaux.

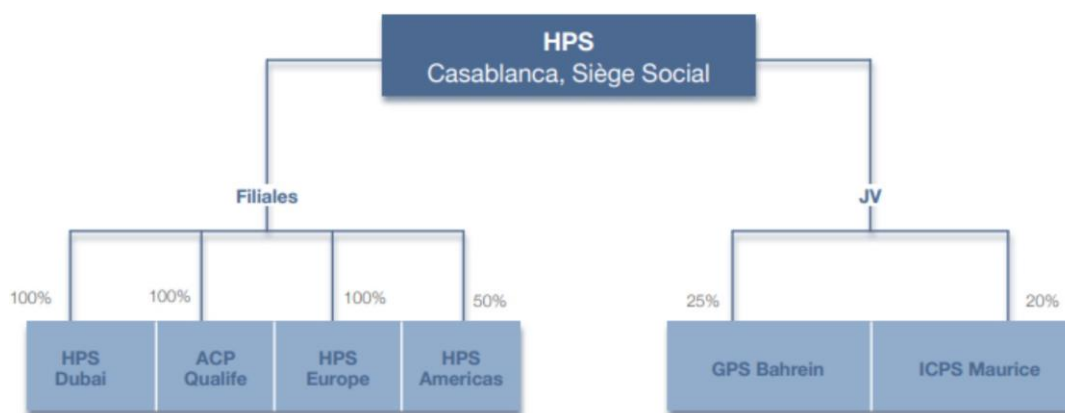


Figure 2 - Actionnariat de HPS dans les entreprises collaboratrices

1.1.2 Historique de HPS

HPS a été fondée en 1995 par un groupe de consultants et d'experts marocains en monétique. La société signe son premier contrat avec le groupement des banques marocaines pour mener une étude sur l'interopérabilité monétique au Maroc. Les collaborateurs d'HPS se donnent ainsi les moyens d'atteindre leur objectif en concevant la solution PowerCARD, résultat d'un travail de longue haleine qui vise à satisfaire les exigences des clients du secteur monétique. En 1996, HPS signe son premier contrat avec la Société générale marocaine des banques (SGMB) et exporte dans la foulée sa solution technologique pour le compte de la banque BKME au Koweït. On peut déjà s'apercevoir de la volonté des dirigeants de conquérir des parts de marché à l'international en ayant une première assise au Maroc. En 2002, c'est au tour du Maroc de se doter d'un centre monétique inter-bancaire et de choisir la solution PowerCARD pour traiter de l'ensemble de ses activités monétiques. Cette même année, HPS fait évoluer sa solution PowerCARD en lançant la version 2 de son logiciel. En 2003, l'entreprise a été certifiée ISO 9001 version 2000 pour l'ensemble de ses activités, HPS compte aujourd'hui 850 collaborateurs dont l'objectif est de concevoir et fournir des solutions complètes, modulaires et intégrées. HPS est cotée à la Bourse de Casablanca depuis 2006 et possède des bureaux dans les principaux centres d'affaires (Afrique, Europe, Asie, Moyen-Orient).

1.1.3 Organigramme de l'entreprise

L'organisation d'HPS se découvre au travers cet organigramme (figure 3) qui présente une vue d'ensemble de la répartition des postes et des fonctions au sein d'HPS.

“ Une organisation adaptée et un management engagé ”

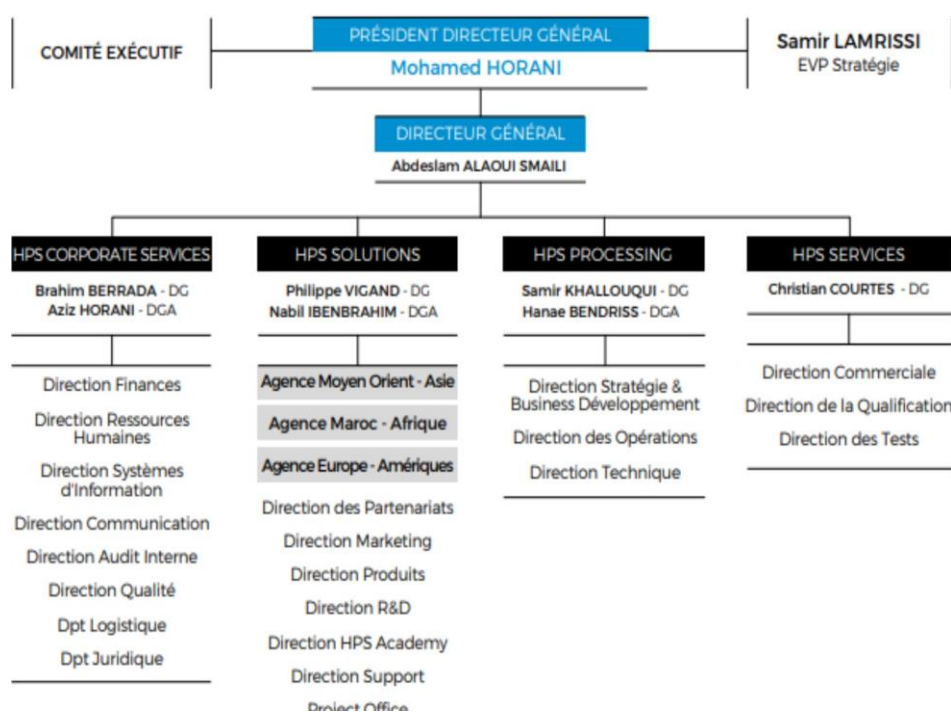


Figure 3 - Organigramme de HPS

1.1.4 Les activités de HPS

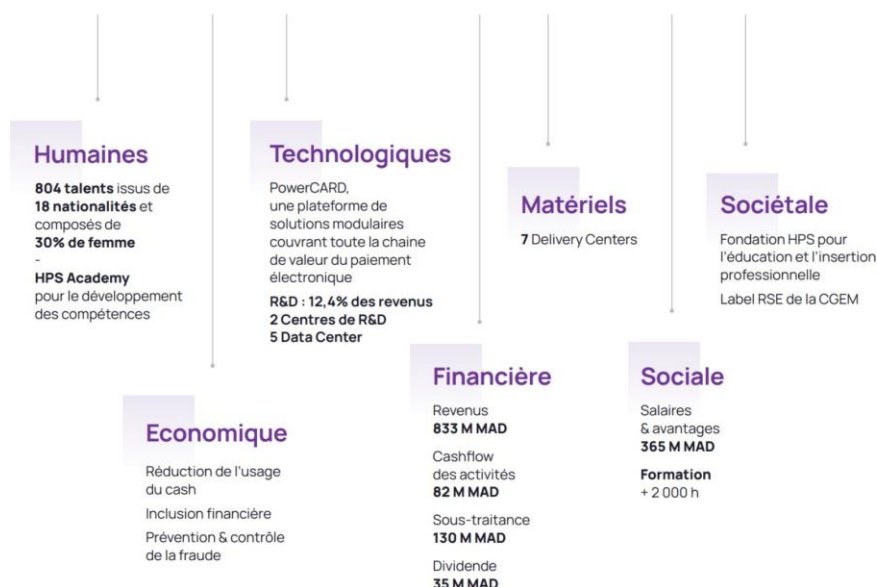


Figure 4 - Les activités de HPS

1.1.5 La solution monétique PowerCARD

PowerCARD, la suite logicielle d'HPS, est la plateforme regroupant les solutions de paiement électronique du Groupe HPS, couvre l'ensemble de la chaîne de valeur du paiement en permettant des paiements innovants grâce à sa plateforme ouverte qui permet le traitement de toutes les transactions initiées par tous les moyens de paiement, provenant de tous les canaux. Elle se distingue par des standards élevés en matière de sécurité, de fluidité et de capacité de traitement des transactions électroniques. La solution PowerCARD proposée par HPS est une solution monétique logicielle intégrée et modulable. -La suite de solutions PowerCARD est utilisée par plus de 350 entités dans plus de 90 pays. -Traite tout type de cartes (crédit, débit, prépayées, entreprise, fidélité, carburant). La figure suivante illustre l'architecture de la solution PowerCard d'HPS.



Figure 5 - Architecture de PowerCARD

PowerCARD a été conçu pour couvrir l'ensemble des transactions (processing) liées à l'activité de paiement. Cette solution est composée de 3 suites logicielles qui assurent les activités de Back, Middle et Front Office.

Front-Office : Cette suite gère le pilotage des terminaux et guichets automatiques. Elle assure notamment les fonctions de sécurité et de vérification, la communication avec les moyens d'acquisition (TPE, GAB, Internet, etc.), la collecte et l'autorisation des transactions ainsi que la gestion des commerçants et des parcs de terminaux d'acquisition, etc.

Middle-Office : Cette deuxième suite assure le lien entre l'acquéreur et l'émetteur pour l'octroi d'autorisations, et le routage dans le cadre d'une interopérabilité nationale et/ou internationale, etc.

Back-Office : C'est la suite qui assure la gestion de l'émission de tous types de cartes (crédit, débit, prépayée, fidélité, etc.), la gestion de la personnalisation des cartes, la gestion de l'acquisition par les commerçants, la compensation (nationale et internationale), la gestion des litiges, le reporting, l'alimentation de la comptabilité des émetteurs et acquéreurs, etc.

1.2 Cadre du projet

Une définition claire du projet est cruciale pour le bon déroulement de celui-ci et pour atteindre les résultats souhaités. Cette section inclut les motivations du projet, sa description ainsi que ses objectifs.

1.2.1 Présentation du projet

Dans l'écosystème des API, la Sandbox joue un rôle crucial en fournissant un environnement contrôlé où les clients et les développeurs peuvent interagir avec les fonctionnalités des API sans perturber les systèmes de production. Actuellement, la Sandbox de ConnectAPI permet aux clients d'explorer et de tester les API en toute sécurité. Cependant, ce système présente des limitations, notamment la nécessité de manipuler directement les fichiers YAML de spécification Swagger pour ajouter de nouvelles documentations, ce qui peut être fastidieux et complexe.

1.2.2 Objectif du Stage

Le projet de stage vise à optimiser la Sandbox en repensant sa fonctionnalité et son interface d'administration. L'objectif est de développer un espace administratif intuitif qui simplifie la gestion des documentations d'API et permet une personnalisation avancée des espaces clients. Cela inclut la création d'une interface graphique flexible et conviviale, permettant aux administrateurs de gérer efficacement les API et d'adapter l'interface aux besoins spécifiques de chaque client.

En améliorant l'expérience utilisateur et en offrant des fonctionnalités de personnalisation avancées, ce projet rendra la plateforme plus efficace et adaptable. Cela permettra de répondre précisément aux exigences et attentes de chaque client, tout en optimisant la robustesse et le dynamisme de l'application.

1.2.3 Missions et tâches

Dans le cadre de notre projet, les missions et tâches sont :

Analyse approfondie des besoins des utilisateurs et des clients : Cette tâche implique l'identification précise des limitations majeures de l'actuelle Sandbox ConnectAPI. Elle comprend également une étude détaillée des pratiques existantes en matière de documentation des exigences fonctionnelles. Enfin, l'objectif est d'orienter le développement de nouvelles fonctionnalités en fonction des besoins exprimés.

Conception d'une interface d'administration intuitive : En collaboration avec l'équipe de développement, ce travail consiste à élaborer des maquettes et des prototypes pour une interface d'administration optimisée. La priorité est de créer une interface utilisateur intuitive pour une navigation fluide. De plus, l'accent est mis sur la facilitation de l'ajout et de la modification graphique des documentations d'APIs.

Personnalisation de la bibliothèque Swagger-UI pour un environnement interne performant : Cette mission implique l'adaptation de la bibliothèque Swagger-UI aux besoins spécifiques de la Sandbox. Cela comprend l'ajout de fonctionnalités supplémentaires pour une meilleure interaction avec les APIs et l'optimisation de l'expérience utilisateur pour une utilisation interne efficace.

Garantie de la qualité et de la fiabilité de l'application : Ce volet garantit la qualité et la fiabilité de l'application par la rédaction et l'exécution rigoureuses de tests unitaires et d'intégration. Ce processus permet une validation méticuleuse du fonctionnement de l'application.

Documentation complète et accompagnement des utilisateurs : Cette tâche prévoit la création de guides utilisateur détaillés pour une prise en main rapide de l'application. De plus, l'organisation de sessions de formation pour les administrateurs et les clients permet un transfert efficace des connaissances pour une utilisation optimale de l'application.

1.3 Dossier de pilotage

L'objectif de cette section est de présenter la démarche de la réalisation du projet en termes d'étapes et d'espace temporel.

1.3.1 Environnement et Méthodologie de travail

En ingénierie logicielle, les pratiques agiles mettent en avant la collaboration entre des équipes auto-organisées et pluridisciplinaires et leurs clients. Elles s'appuient sur l'utilisation d'un cadre méthodologique léger mais suffisant centré sur l'humain et la communication.

1.3.2 La méthode Agile

Les méthodes agiles (ou "agile methodologies" en anglais) regroupent différentes approches de gestion de projet qui mettent l'accent sur la flexibilité, l'adaptabilité et la réactivité face aux changements. Les approches agiles permettent de s'adapter rapidement aux

changements de priorités, aux nouveaux besoins des clients ou aux conditions du marché. Cela est particulièrement important dans les environnements en constante évolution, où les exigences ne sont pas toujours clairement définies au départ. Ainsi, elles favorisent une collaboration étroite entre les équipes de développement, les clients et les parties prenantes.

La méthodologie Agile se base sur ce principe simple : “ planifier la totalité de votre projet dans les moindres détails avant de le développer est contre-productif ”. En effet, la méthode Agile recommande de se fixer des objectifs à court terme. Le projet est donc divisé en plusieurs sous-projets. Une fois l’objectif atteint, on passe au suivant jusqu’à l’accomplissement de l’objectif final.

Autre point important : la méthode Agile repose sur une relation privilégiée entre le client et l’équipe projet. La satisfaction du client étant la priorité.

L’approche Agile offre une plus grande flexibilité et une meilleure visibilité dans la gestion du projet. A notre époque où la personnalisation est si importante, cette méthodologie fait de plus en plus d’adeptes.

1.3.3 La méthode Scrum

Scrum est un Framework avec lequel les utilisateurs peuvent résoudre des problèmes d’adaptation complexes tout en proposant de manière productive et créative des produits de la plus haute valeur possible. C’est l’opposé d’une grande collection de composants obligatoires entrelacés. Scrum n’est pas une méthodologie. Scrum met en œuvre la méthode scientifique de l’empirisme. Scrum remplace une approche algorithmique programmée par une approche heuristique, dans le respect des personnes et l’auto-organisation, pour faire face à l’imprévisibilité et résoudre des problèmes complexes.



Figure 6 - Vue globale de la méthode SCRUM.

1.3.4 Pourquoi Scrum ?

La méthode Scrum a été choisie pour ce projet pour plusieurs raisons. Tout d'abord, Scrum est une méthodologie agile qui est particulièrement adaptée au développement de

logiciels, car elle permet une flexibilité accrue et une collaboration étroite entre les membres de l'équipe. Elle permet également une livraison rapide et continue des fonctionnalités, ce qui est important dans un projet de développement de logiciels. De plus, Scrum est une méthode itérative et incrémentale, ce qui signifie que le projet est divisé en plusieurs itérations ou sprints, avec des livraisons de fonctionnalités à chaque sprint. Cette approche permet de s'assurer que les fonctionnalités les plus importantes et les plus prioritaires sont livrées en premier.

Et que le projet peut s'adapter rapidement aux changements de priorités ou aux nouveaux besoins. Enfin, Scrum encourage une forte collaboration entre les membres de l'équipe, ce qui est important dans un projet complexe impliquant plusieurs équipes et parties prenantes. Il permet également une transparence et une communication ouverte, ce qui est essentiel pour assurer la réussite du projet.

1.3.5 Planification du projet

Pour illustrer le processus de réalisation du projet, nous utilisons un fichier Excel pour représenter l'état d'avancement des différentes tâches du projet. Ce fichier inclut les Sprints, leur description, et leur statut d'avancement.

Sprint #	Task	Task Description	est	statu
0	Étude existant, analyse des besoins et des exigences, Benchmark, configuration de l'environnement & Conception de l'architecture	Effectuer une analyse approfondie des besoins et des exigences du projet pour la mise à niveau de la Sandbox.	5	DONE
1 & 2	Sandbox UI: product & product release	Implémentation du modèle de données : (Points d'accès RestApi / Opérations CRUD), création d'interfaces graphiques pour afficher les produits et leurs versions.	10	DONE
3	Sandbox UI : Solution & Module	Implémentation du modèle de données : (Points d'accès RestApi / Opérations CRUD), création d'interfaces graphiques pour afficher les modules et les solutions.	5	DONE
4	Sandbox UI : Integration of all Screen layouts & components	Intégrer la navigation Workflow et le contrôle sur la disposition des composants (enable/disable, visible/hidden) dynamiquement depuis la base de données.	10	DONE
5	Sandbox Admin: User Managemenet	Création d'un espace administratif sécurisé (sandbox-ui-admin) pour gérer les autorisations des utilisateurs (droits d'accès).	10	DONE
6	Sign up & Authentication	Ajout de l'inscription et de l'authentification.	10	DONE
7 & 8 & 9	Sandbox UI LIB: Customise Swagger UI, Generation of dynamic OpenAPI Documentation	Personnalisation de la bibliothèque Swagger-UI pour répondre aux besoins spécifiques de la documentation OPEN API, et automatiser la génération de la doc depuis un fichier yaml dynamique.	20	50%
10	Sandbox Admin: manage Api, Module, solution ...	Interface graphique pour publier dynamiquement une nouvelle API, version, module, solution via une interface d'administration.	10	10%
11	Sandbox Admin : monitoring and reporting	Suivi de toutes les activités des utilisateurs, logs, opérations fréquentes, vérification de l'état de santé de l'application.	10	
12	Sandbox UI: Guide & how to?	Intégration d'une section d'aide pour guider les clients à utiliser l'application.	10	
12	Tests	Tests fonctionnels et d'intégration pour identifier et corriger les bugs et les problèmes.	5	
13	Déploiement et Maintenance	Déploiement de l'application en production et validation de son fonctionnement.	10	
14	Documentation et finalisation	Préparation de la version finale de l'application pour la présentation et l'évaluation.	5	
			120	

Figure 7 - Sprints par fichier Excel

1.3.5.1 Diagramme de Gantt

Le diagramme de Gantt, largement utilisé en gestion du projet, est un outil extrêmement efficace pour représenter visuellement l'état d'avancement des activités qui composent un projet. Il offre une vue d'ensemble en un coup d'œil, permettant de :

- Identifier les différentes tâches à prendre en compte ;
- Visualiser les dates de début et de fin de chaque tâche ;

- Estimer la durée prévue pour chaque tâche ;
- Repérer les éventuels chevauchements de tâches et leur durée ;
- Déterminer les dates de début et de fin du projet dans son ensemble.

Grâce à sa représentation graphique claire, le diagramme de Gantt facilite la planification, la coordination et le suivi des projets, offrant ainsi un moyen pratique de gestion pour les équipes et les parties prenantes. Son utilisation permet de visualiser rapidement l'état d'avancement global du projet et de prendre des décisions éclairées pour optimiser la progression.

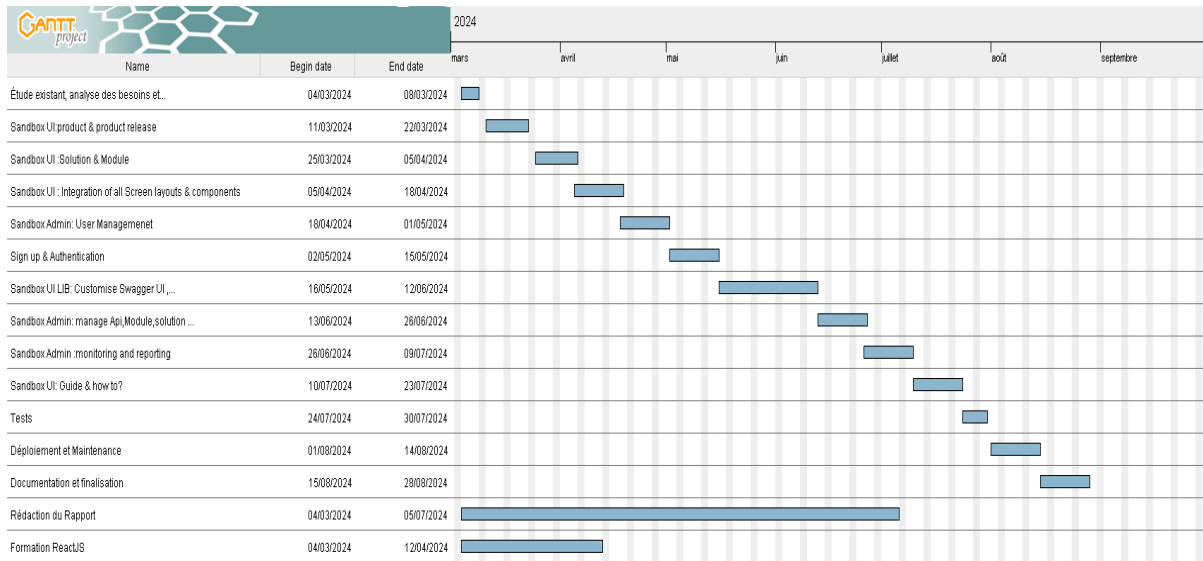


Figure 8 - Diagramme de Gantt

Conclusion

Le premier chapitre présente un point de départ pour l'élaboration du projet. Il définit son contexte général, l'organisme d'accueil, le cadre général et finalement il présente l'approche adoptée pour mener le projet à bien. Dans le chapitre suivant, nous allons présenter la phase de l'étude de l'existant, la problématique et finalement le périmètre du projet.

Chapitre 2

Spécification et analyse

Ce chapitre détaille la méthodologie employée dans notre projet de fin d'études. Après avoir présenté le contexte organisationnel, nous examinerons les étapes clés et les phases de développement. Nous aborderons également les défis rencontrés et les solutions apportées, offrant ainsi une vue d'ensemble du processus suivi.

2 Spécification et analyse

2.1 Etudes de l'existant

Nous avons mené une étude de l'existant en analysant différentes plateformes de gestion de Sandbox API disponibles sur le marché. Cette analyse approfondie nous a permis de comprendre les fonctionnalités existantes ainsi que les bonnes pratiques appliquées dans ce domaine spécifique. Nous avons également exploré d'autres solutions similaires disponibles à un coût potentiellement plus abordable.

Nous avons constaté que la Sandbox ConnectAPI actuelle, bien qu'utile pour fournir un environnement contrôlé aux développeurs et aux clients pour explorer et tester les APIs, présente plusieurs limitations. La gestion des documentations d'API nécessite actuellement une manipulation directe des fichiers YAML de spécification Swagger, ce qui peut être fastidieux et source d'erreurs. De plus, l'interface utilisateur manque de personnalisation, limitant ainsi sa flexibilité pour répondre aux besoins spécifiques des différents utilisateurs et clients.

Après avoir évalué ces aspects et pris en compte les options disponibles sur le marché, nous avons conclu que la refonte de la Sandbox ConnectAPI est nécessaire pour améliorer son efficacité opérationnelle, augmenter sa flexibilité et offrir une expérience utilisateur enrichie. Nous envisageons donc de développer une nouvelle interface d'administration basée sur des technologies modernes telles que React.js, Spring Boot, en intégrant des fonctionnalités graphiques avancées pour la gestion des documentations d'API. Cette approche garantira la fiabilité de la plateforme tout en répondant pleinement aux exigences spécifiques de nos clients et utilisateurs.

2.2 Problématique

Dans l'écosystème des API, la Sandbox de ConnectAPI joue un rôle crucial en permettant aux développeurs et aux clients de tester et d'interagir avec les API dans un environnement contrôlé. Cependant, l'infrastructure actuelle présente plusieurs limitations qui impactent négativement l'expérience utilisateur. La gestion des documentations d'API nécessite la manipulation directe des fichiers YAML de spécification Swagger, un processus non seulement fastidieux, mais également sujet aux erreurs. De plus, la personnalisation de l'interface graphique pour répondre aux besoins spécifiques des différents clients est limitée, réduisant ainsi la flexibilité et l'efficacité de la plateforme.

2.3 Solution

Pour résoudre cette problématique, nous allons concevoir une interface d'administration intuitive et personnalisable qui simplifie la gestion des documentations d'API et permet une personnalisation avancée des espaces clients. Une telle interface offrirait non seulement une meilleure expérience utilisateur, mais augmenterait également l'efficacité opérationnelle des administrateurs de la plateforme.

En repensant l'interface et les fonctionnalités de la Sandbox, le projet vise à créer une solution qui répond aux besoins évolutifs des clients et des développeurs. L'intégration de fonctionnalités graphiques pour la gestion des APIs, ainsi que la personnalisation de la bibliothèque Swagger-UI, permettront de surmonter les limitations actuelles, offrant ainsi une plateforme plus flexible, robuste et adaptée aux exigences spécifiques de chaque client.

2.4 Périmètre du projet

Pour affiner la problématique de mon projet de fin d'études, j'ai employé la méthode QQOQCP pour appréhender plus précisément les éléments interagissant avec celle-ci.

	Questions	Réponses
Qui ?	À qui cela s'adresse ?	Administrateurs de la plateforme ConnectAPI, développeurs utilisant les APIs, et clients interagissant avec les fonctionnalités des APIs.
Quoi ?	À propos de quoi s'agit-il ?	Redéfinir la Sandbox ConnectAPI en développant une interface d'administration intuitive et personnalisable.
Où ?	Où réside le problème ?	La manipulation directe des fichiers YAML de spécification Swagger est fastidieuse, et la personnalisation de l'interface est limitée, réduisant la flexibilité de la plateforme.
Quand ?	À partir de quand ?	Le besoin est apparu avec l'augmentation de la complexité et des exigences des clients utilisant la Sandbox ConnectAPI.
Comment ?	De quelle façon procéder ?	Développer une interface d'administration conviviale et personnalisable, intégrer des fonctionnalités graphiques pour la gestion des documentations d'APIs, et personnaliser la bibliothèque Swagger-UI.
Pourquoi ?	Dans quelle finalité ?	Pour améliorer l'expérience utilisateur, rendre la gestion des APIs plus simple et flexible, et permettre une personnalisation avancée des espaces clients.

Tableau 1 - La méthode QQOQCP

2.4.1 Analyse des besoins

Cette section expose les besoins fonctionnels et non fonctionnels, accompagnés d'un diagramme de cas d'utilisation.

2.4.1.1 *Besoins fonctionnels*

Pour ce projet de développement de la Sandbox ConnectAPI, nous avons identifié trois modules distincts, chacun répondant à des besoins spécifiques et contribuant à une solution intégrée efficace :

Sandbox UI : Ce module servira de portail public destiné aux clients de la solution PowerCARD et les collaborateurs internes et externes. Son objectif principal sera de présenter de manière claire et intuitive les API disponibles, permettant aux utilisateurs de les comprendre et de les tester facilement.

Sandbox Admin : Conçu comme un portail privé d'administration, ce module sera dédié à la gestion interne de la Sandbox ConnectAPI. Les fonctionnalités clés incluront la gestion des utilisateurs, la gestion des produits, la gestion des versions des produits, la gestion des modules, la gestion des solutions, ainsi que la gestion détaillée des APIs. Cela permettra aux administrateurs d'assurer un contrôle complet sur l'environnement de la Sandbox et de ses composants.

Sandbox Lib : Ce module se concentrera sur la personnalisation avancée de la bibliothèque Swagger-UI. L'objectif est de personnaliser cette interface pour répondre précisément aux besoins internes de la Sandbox ConnectAPI. Cela comprendra l'intégration de fonctionnalités spécifiques et la configuration visuelle de Swagger-UI afin d'optimiser l'expérience utilisateur tout en maintenant une flexibilité maximale pour les ajustements futurs.

Le développement de ces trois modules distincts et interopérables nous permet de créer une solution efficace et adaptable pour la gestion et l'utilisation des API au sein de la Sandbox ConnectAPI.

2.4.1.2 *Besoins non fonctionnels*

Performance : Assurer des temps de réponse rapides de l'application même en cas de charges élevées.

Précision des résultats : Garantir un niveau élevé de précision dans les résultats produits par l'application afin de minimiser les erreurs et d'assurer la fiabilité des données fournies aux utilisateurs.

Utilisation des ressources : Optimiser l'utilisation des ressources matérielles et logicielles de l'application pour minimiser la consommation de mémoire, de puissance de calcul et de bande passante, tout en maintenant des performances élevées.

Réutilisabilité du code : Concevoir l'application de manière modulaire et réutilisable afin de faciliter le développement de nouvelles fonctionnalités et de réduire le temps et les efforts nécessaires pour les intégrer dans le système existant.

Conclusion

Dans ce deuxième chapitre, nous avons posé les bases pour notre projet. En identifiant la problématique, définissant les objectifs et planifiant les étapes, nous avons tracé la voie à suivre pour la réalisation de notre solution. Ces éléments guideront nos actions à venir vers la concrétisation de notre projet.

Chapitre 3

Analyse & conception

Après avoir terminé les phases de spécifications et d'analyse, nous entamons l'étude conceptuelle. Pour commencer, on va aborder la conception préliminaire du système en se basant sur les fonctionnalités identifiées, avant de passer à la conception détaillée.

3 Étude conceptuelle

Cette section présente plusieurs scénarios d'utilisation du système, ainsi que les diagrammes de classes et de cas d'utilisation.

3.1 Formalisme UML



Figure 9 - Uml logo

UML, (Unified Modeling Language), que l'on peut traduire par "langage de modélisation unifié" est une notation permettant la modélisation d'un problème. Ce langage est né de la fusion de plusieurs méthodes existantes auparavant, et est devenu la référence en termes de modélisation objet. Entre 1910 et 1990, de nombreux analystes ont mis au point des approches orientées objets, ainsi en 1994 il existait plus de 50 méthodes objet. Toutefois seules 3 méthodes ont véritablement émergé :

- La méthode OMT de Rumbaugh
- La méthode BOOCH'93 de Booch.
- La méthode OOSE de Jacobson (Object Oriented Software Engineering)

A partir de 1994. Rumbaugh et Booch (rejoints en 1995 par Jacobson) ont uni leurs efforts pour mettre au point la méthode unifiée incorporant les avantages de chacune des méthodes précédentes. La méthode unifiée à partir de la version 1.0 devient UML (Unified Modeling Language), une notation universelle pour la modélisation objet. UML 1.0 est soumise à l'OMG1 en janvier 1997 mais elle ne sera acceptée qu'en novembre 1997 dans sa version 1.1, date à partir de laquelle UML devient un standard international cette méthode représente un moyen de spécifier, représenter et construire les composantes d'un système informatique. En effet, la notation unifiée définit 9 diagrammes pour représenter les différents points de vue de modélisation. Ces diagrammes permettent de visualiser et de manipuler les éléments de modélisation. Les diagrammes définis par UML, sont les suivants :

Les diagrammes de cas d'utilisation : représentation des fonctions du système du point de vue de l'utilisateur.

Les diagrammes de séquence : représentation temporelle des objets et de leurs Les diagrammes d'activités représentation du comportement d'une opération en termes d'actions.

Les diagrammes d'activités : représentation graphique du comportement d'une méthode ou le déroulement d'un cas d'utilisation.

Les diagrammes de classes : représentation de la structure statique en termes de classes et de relations.

La conception est la plus importante étape du cycle du développement d'application. Elle se base essentiellement sur la bonne spécification et l'analyse des besoins. A présent, nous sommes dans la phase de concevoir la solution, et définir les composants globaux de notre projet.

3.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation est un outil de modélisation utilisé pour représenter les interactions entre les différents utilisateurs et le système. Il permet de décrire comment les utilisateurs interagissent avec le système pour réaliser une tâche ou une fonctionnalité spécifique.

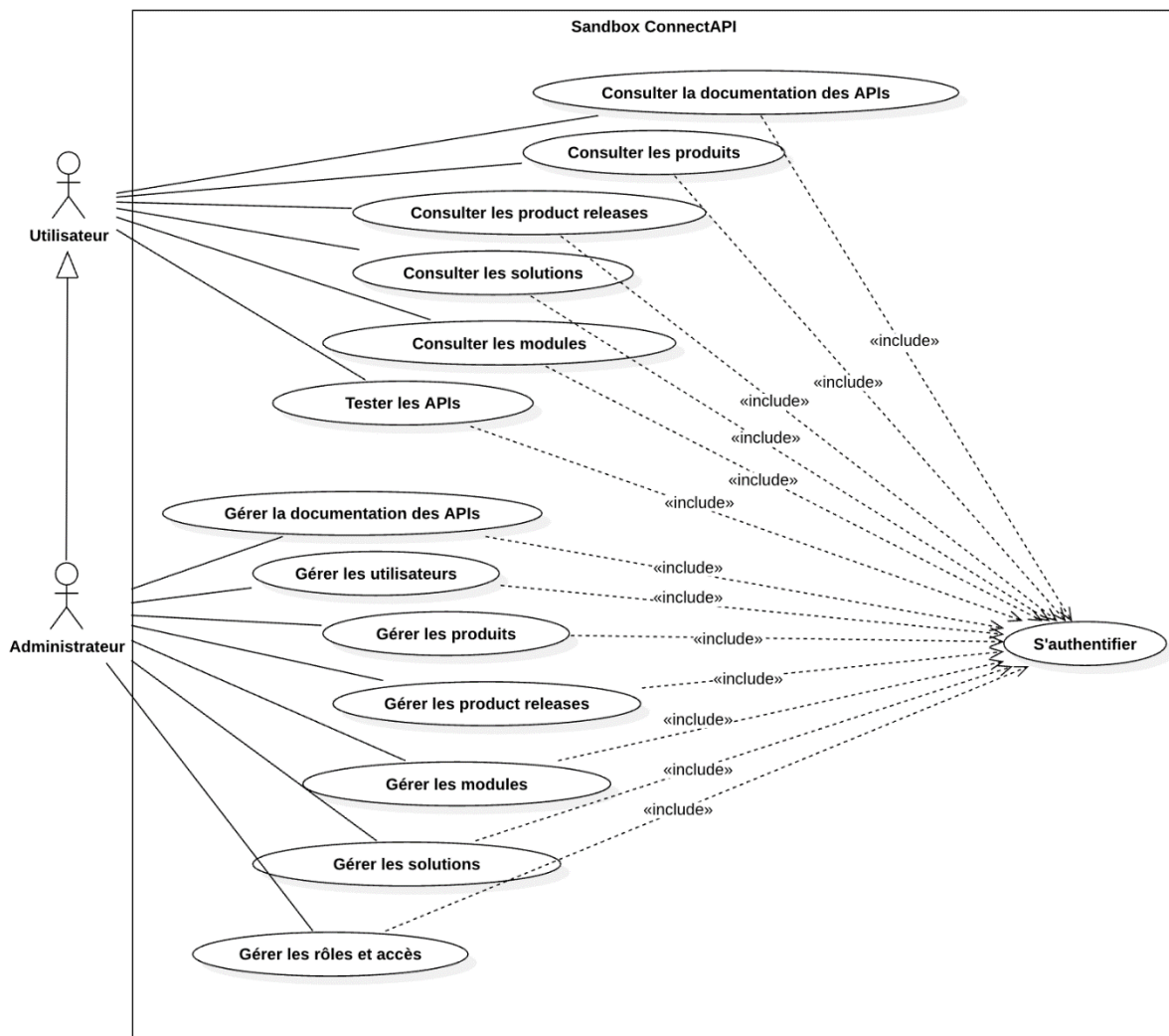


Figure 10 - Diagramme de Cas d'Utilisation global de la Sandbox ConnectAPI HPS

3.2.1 Définition des acteurs

Dans le contexte de la modélisation des systèmes, un acteur est une représentation d'une entité externe au système étudié, telle qu'un utilisateur humain, un dispositif matériel ou encore un autre système. L'acteur est impliqué dans un scénario d'utilisation et joue un rôle en interagissant directement avec le système.

Les acteurs qui interagissent avec la Sandbox ConnectApi sont:

- **Utilisateur** : Collaborateur interne ou externe qui utilise la Sandbox ConnectAPI pour explorer et tester les APIs disponibles.
- **Administrateur** : Responsable de l'administration et de la gestion de la Sandbox ConnectAPI.

3.2.2 Description des cas d'utilisation

L'administrateur, en tant que gestionnaire du système, dispose d'un éventail de privilèges plus large, incluant la gestion des utilisateurs, la gestion des produits et des versions, la gestion des modules, la gestion des solutions, la gestion de la documentation des API et la gestion des rôles et des accès. L'utilisateur, quant à lui, peut accéder aux informations et fonctionnalités de base du système, telles que la consultation des produits, des versions de produits, des solutions, des modules, de la documentation des API et le test des API.

L'authentification est obligatoire tant pour l'administrateur que pour l'utilisateur afin d'accéder au système, garantissant ainsi que seuls les individus autorisés peuvent utiliser ses ressources et ses fonctionnalités. Ce diagramme décrit efficacement les rôles d'utilisateur du système et leurs fonctionnalités correspondantes.

3.3 Diagramme de séquence

Le diagramme de séquence est un outil de modélisation utilisé pour représenter l'interaction dynamique entre les différents objets d'un système. Il permet de décrire comment les objets interagissent les uns avec les autres dans un processus ou un scénario donné.

3.3.1 Diagramme de Séquence du cas de Test des APIs par l'utilisateur

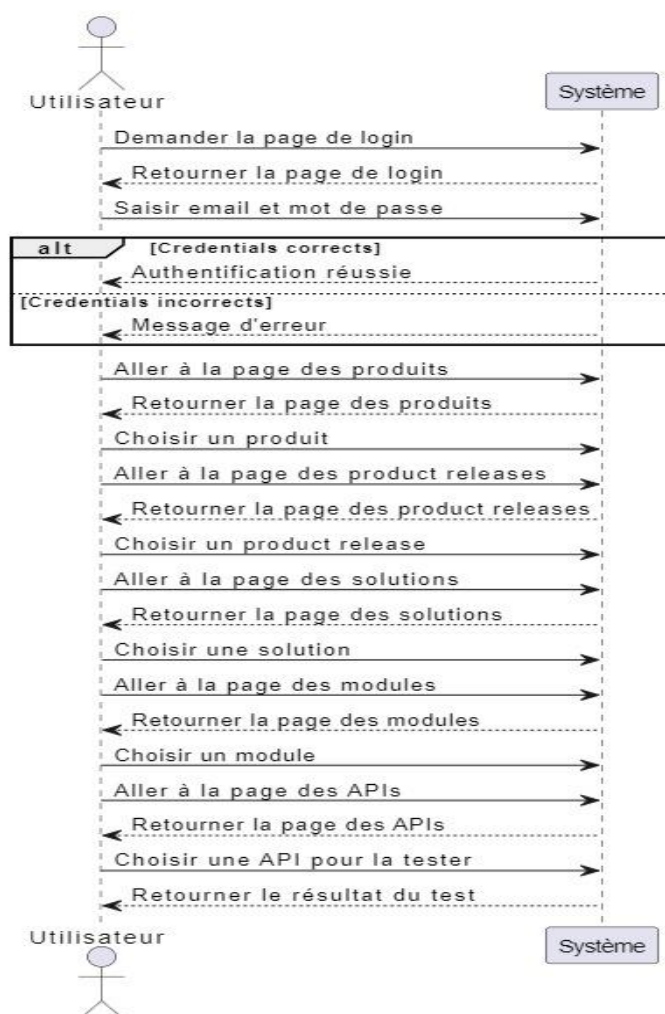


Figure 11 - Diagramme de Séquence du cas de Test des APIs par l'utilisateur

Ce diagramme de séquence illustre comment un utilisateur interagit avec le système pour explorer et tester des APIs, à travers plusieurs étapes structurées. Initialement, l'utilisateur accède à l'application et demande la page de login. Le système répond en lui fournissant l'interface de login où il saisit ses informations d'identification. Après la soumission, le système vérifie ces informations pour l'authentifier. Si l'authentification réussit, l'utilisateur navigue à travers les différentes fonctionnalités de l'application. Il commence par consulter les produits disponibles, puis les versions spécifiques de ces produits, suivies par la sélection d'une solution et d'un module. À chaque étape, le système retourne les informations pertinentes sur les choix disponibles et permet à l'utilisateur de sélectionner les éléments souhaités. Ensuite, l'utilisateur accède à la documentation des APIs fournies par l'entreprise. Une fois la documentation consultée, il choisit une API spécifique à tester. Le système facilite ce processus en fournissant un environnement de test où l'utilisateur peut exécuter des scénarios pour évaluer le comportement de l'API sélectionnée. À chaque étape de ce parcours, le système répond de

manière interactive aux actions de l'utilisateur, assurant une expérience utilisateur fluide et informative.

3.3.2 Diagramme de Séquence pour l'Administrateur : Cas d'Activation d'un Compte

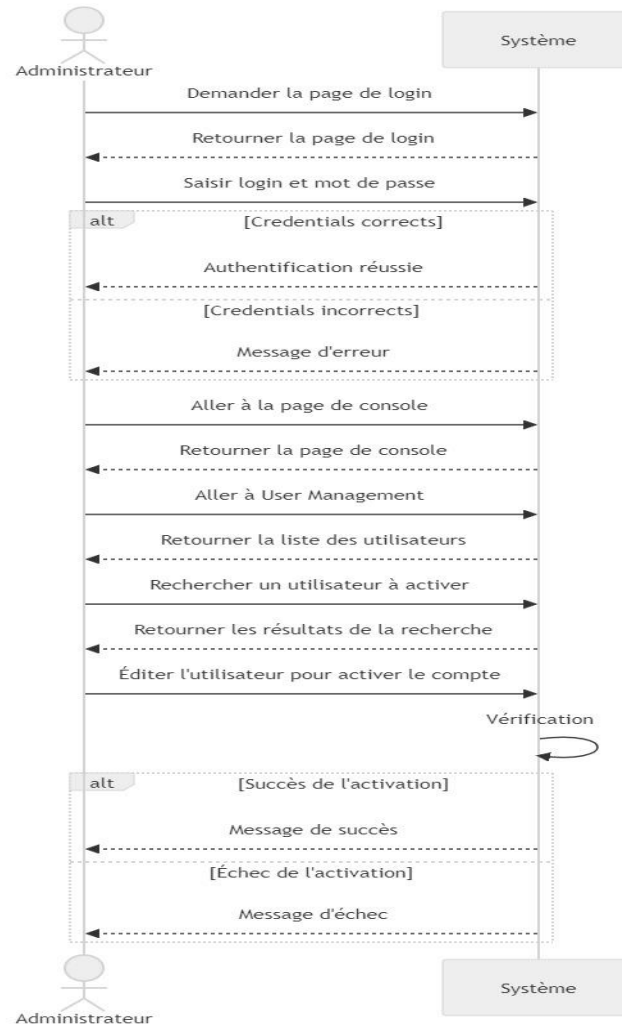


Figure 12 - Diagramme de Séquence pour l'Administrateur : Cas d'Activation d'un Compte Utilisateur

Ce diagramme de séquence détaille les interactions complexes entre un administrateur et le système dans le contexte de la gestion des utilisateurs. Initialement, l'administrateur demande l'accès à l'application et entre ses informations d'identification pour se connecter. Le système vérifie ces informations pour autoriser l'accès à la console d'administration. Une fois connecté, l'administrateur accède au module de gestion des utilisateurs. Il commence par consulter la liste complète des utilisateurs enregistrés dans le système. Pour gérer les comptes utilisateur, l'administrateur utilise une fonction de recherche pour localiser un utilisateur spécifique parmi ceux enregistrés. Une fois l'utilisateur trouvé, l'administrateur prend des mesures pour activer son compte. Le système vérifie alors les données saisies par l'administrateur pour s'assurer de leur exactitude et de leur validité. Si les informations sont correctes et que l'activation réussit, le système confirme cette action et informe l'administrateur

du succès. En cas d'échec, le système retourne un message explicatif indiquant les raisons du problème rencontré. Cette séquence d'actions illustre comment le système facilite la gestion efficace des utilisateurs par l'administrateur, en garantissant la sécurité et la conformité des opérations effectuées.

3.4 Diagramme de classe

Le diagramme de classe est un outil de modélisation utilisé pour représenter la structure statique d'un système orienté objet. Il permet de décrire les classes, les attributs, les méthodes et les relations entre les différentes classes.

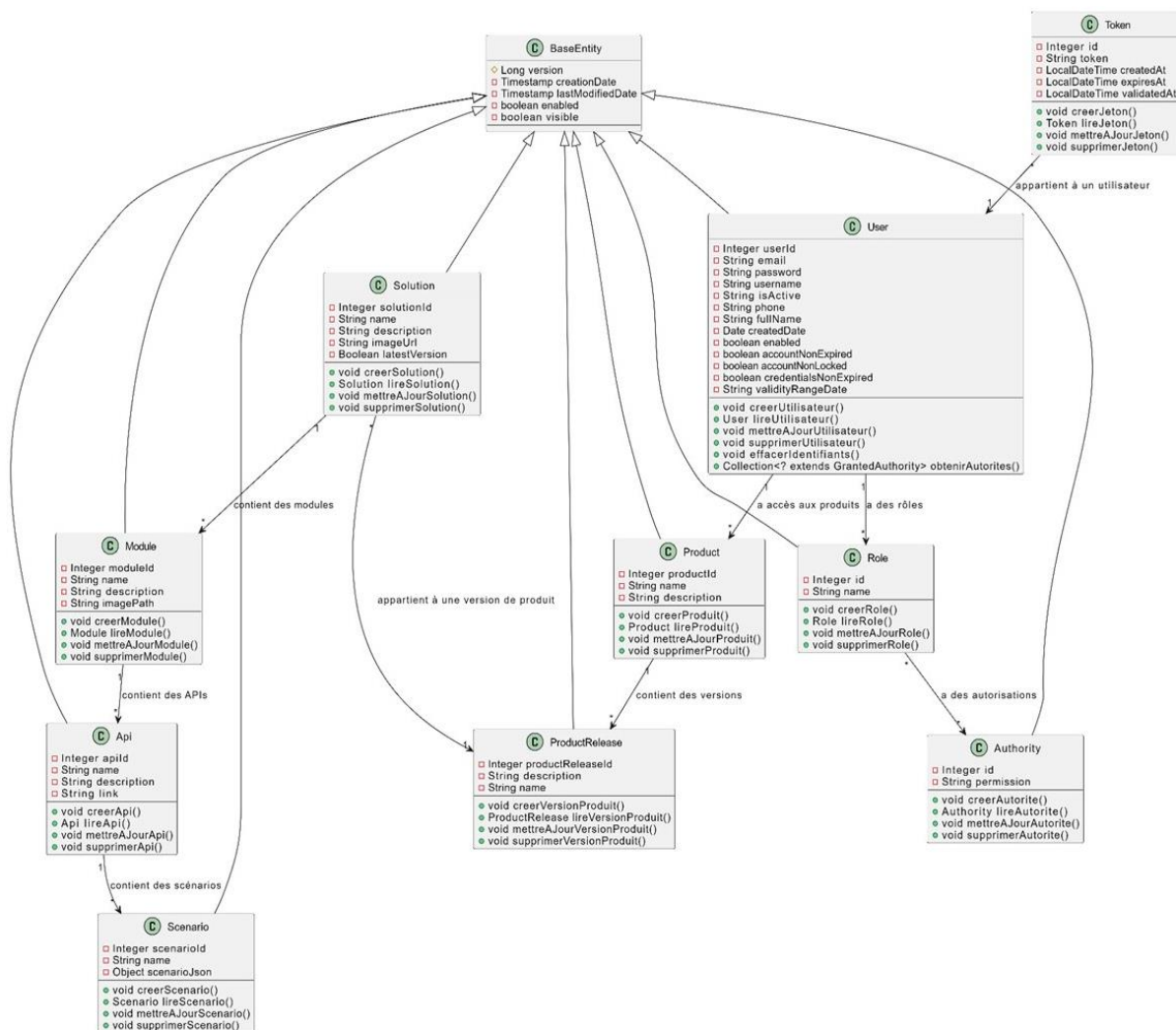


Figure 13 - Diagramme de class

Le diagramme de classes proposé pour ce système illustre la structure organisationnelle et fonctionnelle des entités clés et de leurs relations. Chaque classe principale hérite des propriétés de la classe abstraite BaseEntity, incluant des attributs comme la version, les dates de création et de dernière modification, ainsi que des indicateurs d'état comme l'activation et la visibilité. Par exemple, la classe Product encapsule les informations spécifiques à un produit telles que son nom et sa description, tandis que la classe ProductRelease représente les versions

spécifiques de ce produit. Cette hiérarchie permet une gestion efficace des produits et de leurs versions tout au long du cycle de vie applicatif.

- **Gestion des Accès et Autorisations** : Chaque **User** est associé à plusieurs **Product** à travers une relation d'accès. Les **Rôles** attribués aux utilisateurs définissent leurs permissions spécifiques via des **Authority**, régulant ainsi les actions permises dans le système. Par exemple, un administrateur pourrait avoir accès à toutes les fonctionnalités et données, tandis qu'un utilisateur standard pourrait être restreint à certaines opérations.
- **Traçabilité et Sécurité des Sessions** : Les **Token** sont utilisés pour gérer l'authentification des utilisateurs. Chaque token est associé à un utilisateur spécifique et contient des informations de création, d'expiration et de validation (`createdAt`, `expiresAt`, `validatedAt`), garantissant ainsi la sécurité des sessions actives et la prévention des accès non autorisés.
- **Structuration Modulaire** : Les Solutions regroupent plusieurs **Modules**, qui à leur tour contiennent des **Api** et des **Scenarios**. Cette structure modulaire permet une gestion flexible et évolutive des fonctionnalités offertes par le système, facilitant ainsi le développement, la maintenance et l'extension des capacités applicatives.

Ce diagramme de classes vise à fournir une représentation claire et organisée des entités et de leurs interactions, soutenant ainsi une implémentation robuste et sécurisée du système applicatif développé.

Conclusion

Ce Chapitre a été consacré à la partie modélisation du système en se basant sur les spécifications détaillées par le métier. Dans le prochain chapitre nous aborderons la partie technique qui présente les architectures implémentées et les techniques choisies pour le développement du projet.

Chapitre 4

Architecture technique

Après avoir établi les bases de la conception préliminaire dans le chapitre précédent, nous passons maintenant à l'architecture technique et aux outils nécessaires pour la mise en œuvre du système. Ce chapitre détaillera les différents outils et technologies sélectionnés pour assurer une réalisation efficace et conforme aux spécifications établies.

4 Architecture technique

4.1 Architecture logicielle du système

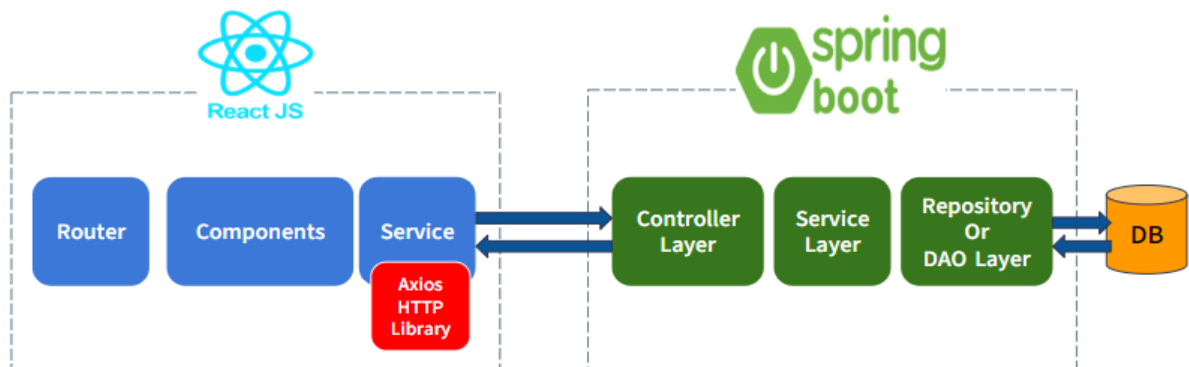


Figure 14 - Architecture technique du système

L'architecture de notre application est conçue en utilisant deux technologies principales: React pour le front-end et Spring Boot pour le back-end. Cette architecture est représentée dans l'image ci-dessus et illustre comment les différentes couches interagissent pour offrir une application web robuste et maintenable.

La partie front-end de l'application est construite avec React, une bibliothèque JavaScript populaire pour la création d'interfaces utilisateur. L'architecture front-end se compose de plusieurs couches :

Front-End : React JS

Router : Le routeur est responsable de la gestion de la navigation à travers l'application. Il mappe les différentes URL aux composants correspondants, permettant une navigation fluide entre les différentes pages et fonctionnalités de l'application.

Components : Les composants représentent les unités de base de l'interface utilisateur. Chaque composant est autonome et gère une partie spécifique de l'interface, ce qui rend le code modulaire et réutilisable. Cela facilite également le développement et la maintenance, car les modifications peuvent être isolées à des composants individuels sans affecter l'ensemble de l'application.

Service : La couche service est utilisée pour gérer la logique de l'application qui nécessite une interaction avec le serveur. Dans notre cas, nous utilisons la bibliothèque Axios pour effectuer des requêtes HTTP. Cette bibliothèque permet une communication efficace avec le back-end, envoyant et recevant des données pour alimenter l'interface utilisateur.

Back-End : Spring Boot

La partie back-end de l'application est développée avec Spring Boot, un framework Java puissant pour la création de services web robustes. L'architecture back-end est divisée en plusieurs couches :

Controller Layer : Cette couche reçoit les requêtes HTTP provenant du front-end. Les contrôleurs traitent les requêtes, exécutent la logique métier nécessaire et renvoient les réponses appropriées au front-end.

Service Layer : La couche service contient la logique métier de l'application. Elle interagit avec les contrôleurs pour traiter les requêtes et avec la couche de persistance pour accéder aux données. Cette séparation de la logique métier permet une organisation claire du code et facilite les tests unitaires.

Repository or DAO Layer : Cette couche gère l'accès aux données. Les repositories ou DAOs (Data Access Objects) sont responsables de la communication avec la base de données, effectuant des opérations de lecture, d'écriture, de mise à jour et de suppression des données.

DB (Database) : La base de données stocke toutes les données persistantes de l'application. Elle est accessible via la couche Repository ou DAO, garantissant ainsi une séparation claire entre la logique métier et l'accès aux données.

4.2 Environnement de Développement

Dans cette section du chapitre, nous présentons en détail les différentes technologies, outils et langages de programmation qui ont été utilisés pour mettre en place le projet. Nous explorons les choix que nous avons faits en termes de logiciels et de matériel, ainsi que les raisons pour lesquelles nous les avons sélectionnés. Nous décrivons également comment ces éléments ont été intégrés pour produire le résultat final.

4.2.1 Choix technologique

Le choix technologique a été effectué en se basant sur les technologies les plus récentes et les plus avancées, en utilisant des critères rigoureux pour garantir que seules les technologies à l'état de l'art ont été sélectionnées.

Ce choix a été effectué par HPS, qui a évalué les différentes technologies disponibles sur le marché pour s'assurer que le système utilise les technologies les plus adaptées à ses besoins. En choisissant les technologies les plus récentes et les plus avancées, nous pouvons répondre aux exigences des besoins non fonctionnels.

4.2.2 Langages et Frameworks

4.2.2.1 JAKARTA EE

JAKARTA EE La plate-forme étend Java Platform, Standard Edition (Java SE) en fournissant une API de mapping objet-relationnel, des architectures distribuées et multitiers, et des services web3. La plate-forme se fonde principalement sur des composants modulaires exécutés sur un serveur d'applications.



Figure 15 - Jakarta EE logo

4.2.2.2 Spring boot

Spring Boot est un Framework Java populaire conçu pour faciliter la création d'applications web. Spring Boot fournit un ensemble d'outils et de bibliothèques préconfigurés pour faciliter le développement rapide et la mise en œuvre d'applications.



Figure 16 - Spring Boot logo

4.2.2.3 Spring data JPA

Spring Data est un ensemble de projets qui fournissent une couche d'accès aux données simplifiée et unifiée pour les applications basées sur Spring.

En implémentant Spring Data, nous pouvons facilement créer et gérer des microservices qui interagissent avec différentes sources de données de manière efficace et cohérente. Spring Data peut être utilisé avec une variété de sources de données, notamment les bases de données SQL, NoSQL (MySQL) et les services REST.



Figure 17 - logo Spring data JPA

4.2.2.4 JavaScript

JavaScript est un langage de programmation de scripts utilisé principalement pour créer des pages web interactives et des serveurs. C'est un langage orienté objet à prototype, ce qui signifie que les principales interfaces et les bases du langage sont fournies par des objets qui ne sont pas des instances de classes. Chaque objet est équipé d'un constructeur permettant de créer ses propriétés et une propriété de prototypage qui permet de créer des objets héritiers personnalisés. Les fonctions sont également des objets de première classe en JavaScript.

Dans ce projet, JavaScript est utilisé pour créer des composants d'interface utilisateur réutilisables qui encapsulent la logique et le style.



Figure 18 - JAVASCRIPT logo

4.2.2.5 ReactJS

ReactJS est une bibliothèque JavaScript open source utilisée pour la construction d'interfaces utilisateur pour les applications web. Il est principalement utilisé pour la création de composants réutilisables et d'applications à page unique (SPA).

ReactJS offre plusieurs avantages qui en font un outil précieux pour le développement web :

- Il offre une encapsulation des composants.
- Il permet une mise à jour en temps réel via Virtual DOM .
- Il offre une modularité : facilement modulaire grâce à la création de composants. Les développeurs peuvent facilement ajouter, modifier ou supprimer des composants sans affecter le reste de l'application.
- Il est compatible avec d'autres technologies : ReactJS est compatible avec de nombreuses autres technologies, notamment Node.js, Redux...

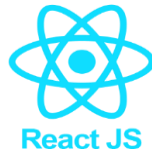


Figure 19 - REACTJS logo

4.2.2.6 Tailwind CSS

Tailwind CSS est un framework CSS moderne et utilitaire qui permet de concevoir des interfaces utilisateur rapidement et de manière efficace. Contrairement aux frameworks CSS traditionnels comme Bootstrap, Tailwind CSS se concentre sur l'utilisation de classes utilitaires pour styliser les éléments HTML, plutôt que d'imposer un ensemble prédéfini de composants.



Tailwind CSS

Figure 20 - Tailwind CSS Logo

4.2.2.7 Gradle

Gradle est un système de gestion de build avancé, open-source et basé sur Groovy et Kotlin. Il simplifie et automatise le processus de build des projets logiciels en permettant aux développeurs de définir les tâches de manière déclarative à l'aide de DSL (Domain-Specific Language). Gradle offre une flexibilité remarquable, une gestion efficace des dépendances, et une extensibilité grâce à ses nombreux plugins, en faisant un choix populaire pour les projets de développement Java et Android.



Figure 21 - Gradle Logo

4.2.2.8 Npm

Npm, abréviation de Node Package Manager, est le gestionnaire de paquets par défaut pour l'environnement d'exécution JavaScript Node.js. Il se compose d'un client en ligne de commande, également nommé npm, et d'une base de données en ligne de paquets publics et privés payants, connue sous le nom de registre npm. npm facilite la gestion des dépendances logicielles en permettant aux développeurs d'installer, de mettre à jour, de partager et de

distribuer des modules JavaScript et des bibliothèques de code réutilisables, essentiels au développement d'applications Node.js.



Figure 22 - Npm Logo

4.2.2.9 JUnit 5

JUnit 5 est la prochaine génération de JUnit. L'objectif est de créer une base à jour pour les tests côté développeur sur la JVM. Cela inclut de se concentrer sur Java 8 et versions ultérieures, ainsi que de permettre de nombreux styles de test différents.



Figure 23 - JUnit 5

4.2.3 Outils de développement

4.2.3.1 Visual studio code

Visual Studio Code (VS Code) est un éditeur de code source léger, open-source et multiplateforme développé par Microsoft. Il offre une expérience de développement riche et personnalisable pour une variété de langages de programmation, y compris JavaScript, TypeScript, Python, C#, et bien d'autres. Doté d'une interface utilisateur moderne et intuitive, VS Code propose des fonctionnalités telles que la coloration syntaxique, la complétion intelligente du code, le débogage intégré, le contrôle de version avec Git, et la prise en charge d'extensions pour étendre ses fonctionnalités. Il offre également des fonctionnalités avancées telles que la refactorisation du code, la recherche et le remplacement avec des expressions régulières, et la gestion des tâches et des configurations de construction. Grâce à son écosystème d'extensions volumineux et actif, les développeurs peuvent personnaliser VS Code selon leurs besoins spécifiques, ce qui en fait un choix populaire parmi les développeurs pour le développement d'applications web, mobiles, et cloud.



Figure 24 - Visual studio code

4.2.3.2 Postman

Postman est une plateforme logicielle complète conçue pour faciliter le développement, le test et la gestion des API. Il offre aux développeurs un environnement intuitif pour créer, envoyer et analyser des requêtes HTTP vers des API. Postman permet non seulement de tester différentes méthodes HTTP telles que GET, POST, PUT et DELETE, mais aussi de vérifier les en-têtes, les paramètres, les corps de requête et les réponses des API. De plus, il propose des fonctionnalités avancées telles que la gestion de collections d'API, la génération de scripts automatisés pour les tests, la documentation d'API interactive et la collaboration au sein d'équipes de développement.



Figure 25 - Postman Logo

4.2.3.3 IntelliJ idea

IntelliJ IDEA est un environnement de développement intégré (IDE) puissant et polyvalent développé par JetBrains. Conçu pour plusieurs langages de programmation, il offre un large éventail de fonctionnalités pour améliorer la productivité des développeurs. IntelliJ IDEA propose une interface utilisateur intuitive, une navigation efficace dans le code, des outils de refactoring avancés, et une intégration transparente avec des systèmes de contrôle de version comme Git. Il comprend des fonctionnalités telles que la complétion intelligente du code, la détection d'erreurs en temps réel, et le débogage avancé pour faciliter le développement. L'IDE prend en charge une grande variété de frameworks et de technologies, avec des plugins disponibles pour étendre ses fonctionnalités selon les besoins spécifiques du projet. Grâce à ses performances élevées et à sa stabilité, IntelliJ IDEA est largement adopté par les développeurs pour le développement d'applications dans divers domaines, des applications web aux applications mobiles et aux logiciels d'entreprise.

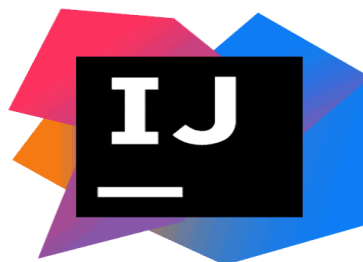


Figure 26 - IntelliJ Idea logo

4.2.3.4 flyway

Flyway est un outil open-source de migration de base de données qui simplifie et automatise le processus de gestion des évolutions de schéma de base de données. Il permet aux développeurs de versionner et de migrer leur base de données de manière cohérente et reproductible à travers différents environnements de développement, test et production. Flyway utilise des scripts SQL simples ou des scripts Java pour définir les changements de schéma, assurant ainsi l'intégrité et la traçabilité des modifications apportées à la base de données. C'est un choix courant pour les équipes de développement souhaitant maintenir une gestion rigoureuse des versions de base de données tout au long du cycle de vie d'une application.



Figure 27 - flyway Logo

4.2.3.5 Bitbucket

Bitbucket est une plateforme de gestion de code source basée sur Git, offrant des fonctionnalités de collaboration et de contrôle de version pour les équipes de développement de logiciels. Propriété d'Atlassian, Bitbucket permet aux utilisateurs de créer des dépôts Git privés ou publics, de gérer des projets de manière efficace, et de collaborer avec d'autres développeurs à travers des pull requests, des commentaires et des intégrations avec des outils de développement tiers. Bitbucket prend en charge les workflows de développement Agile et propose des fonctionnalités avancées telles que l'intégration continue (CI/CD), la gestion des branches, et la sécurité des dépôts avec des permissions granulaires.



Figure 28 - Bitbucket Logo

4.2.3.6 Git

Git est un système de gestion de versions de code source distribué qui permet de suivre les modifications apportées à un projet de développement de logiciel. Il permet aux développeurs de travailler en équipe et de collaborer sur un projet de manière efficace.

Git stocke chaque version du projet sous forme de commit, qui enregistre les modifications apportées aux fichiers du projet. Nous pouvons travailler sur différentes branches

du projet en parallèle, puis fusionner les branches une fois que les modifications ont été testées et validées.



Figure 29 - Git Logo

4.2.3.7 OpenApi

OpenAPI est une spécification standardisée pour la création et la description d'APIs RESTful. Elle permet aux développeurs de définir les points de terminaison d'une API, les méthodes HTTP disponibles, les paramètres d'entrée, les réponses attendues et d'autres aspects techniques de manière cohérente et lisible par les machines. Grâce à OpenAPI, les équipes de développement peuvent automatiser la génération de documentation, de clients et de serveurs, ainsi que faciliter la validation et les tests des APIs. La spécification OpenAPI est largement adoptée dans l'industrie pour améliorer la collaboration et la productivité dans le



Figure 30 - OpenApi Logo

développement d'APIs.

4.2.3.8 Swagger

Swagger est une suite d'outils open-source qui utilise la spécification OpenAPI pour concevoir, documenter et tester des APIs. Les principaux composants de Swagger incluent :

- **Swagger Editor** : Un éditeur en ligne qui permet de créer et d'éditer des fichiers de spécification OpenAPI de manière intuitive.
- **Swagger UI** : Une interface utilisateur interactive qui génère une documentation visuelle et permet de tester les endpoints de l'API en temps réel.
- **Swagger Codegen** : Un générateur de code qui produit des clients et des serveurs dans différents langages de programmation à partir des fichiers OpenAPI.

Swagger est devenu un outil incontournable pour les développeurs souhaitant améliorer la qualité et l'efficacité de leurs APIs.



Figure 31 - Swagger Logo

4.2.3.9 StopLight

Stoplight est une plateforme de gestion d'APIs qui offre une suite d'outils pour la conception, la documentation et le test des APIs. Parmi ses principaux composants, on trouve:

- **Stoplight Studio** : Un éditeur visuel puissant pour créer et modifier des spécifications OpenAPI, avec des fonctionnalités de validation et de prévisualisation en temps réel.
- **Stoplight Documentation** : Permet de générer et d'héberger une documentation interactive et attrayante pour les APIs, facilitant ainsi la compréhension et l'utilisation des APIs par les développeurs.
- **Stoplight Testing** : Offre des outils de test et de simulation pour garantir la conformité des APIs aux spécifications définies.

Stoplight se distingue par son interface utilisateur conviviale et ses fonctionnalités avancées, qui aident les équipes de développement à gérer efficacement le cycle de vie des APIs.



Figure 32 - StopLight LOGO

Conclusion

Dans ce quatrième chapitre, nous avons examiné les outils essentiels à la réalisation de notre solution. En choisissant judicieusement ces ressources, nous avons établi une base pratique pour transformer notre vision en action. Ces outils seront nos alliés tout au long du processus de développement, nous guidant vers la concrétisation de notre projet.

Chapitre 5

Réalisation

Ce chapitre mettra en avant la mise en œuvre des interfaces de notre application en présentant les différents écrans que nous avons développés. Les interfaces jouent un rôle crucial dans l'expérience utilisateur, offrant une interaction intuitive et efficace.

5 Mise en œuvre

Cette section a pour objectif de donner une présentation des écrans du système développé.

5.1 Sandbox-UI

La Sandbox-UI est dédiée aux clients ainsi qu'aux collaborateurs internes et externes, leur permettant de se connecter pour explorer et tester les API développées par HPS. Elle sert de plateforme sécurisée, permettant aux utilisateurs de simuler divers scénarios d'utilisation des API sans impacter l'environnement de production.

5.1.1 L'interface Accueil

Cette interface illustre la page d'accueil de la plateforme Sandbox ConnectApi, destinée aux collaborateurs internes et externes et administrateurs. Elle présente l'API PowerCARD CONNECT', mettant en avant ses avantages d'intégration plus facile, rapide et sécurisée avec les services PowerCARD. Les utilisateurs sont accueillis avec des options claires pour explorer les API disponibles ou demander une démonstration. Cette page d'accueil sert de point d'entrée pour les utilisateurs, leur offrant une vue d'ensemble des fonctionnalités et opportunités offertes par la plateforme, tout en les invitant à approfondir leur exploration et utilisation des services de la sandbox.

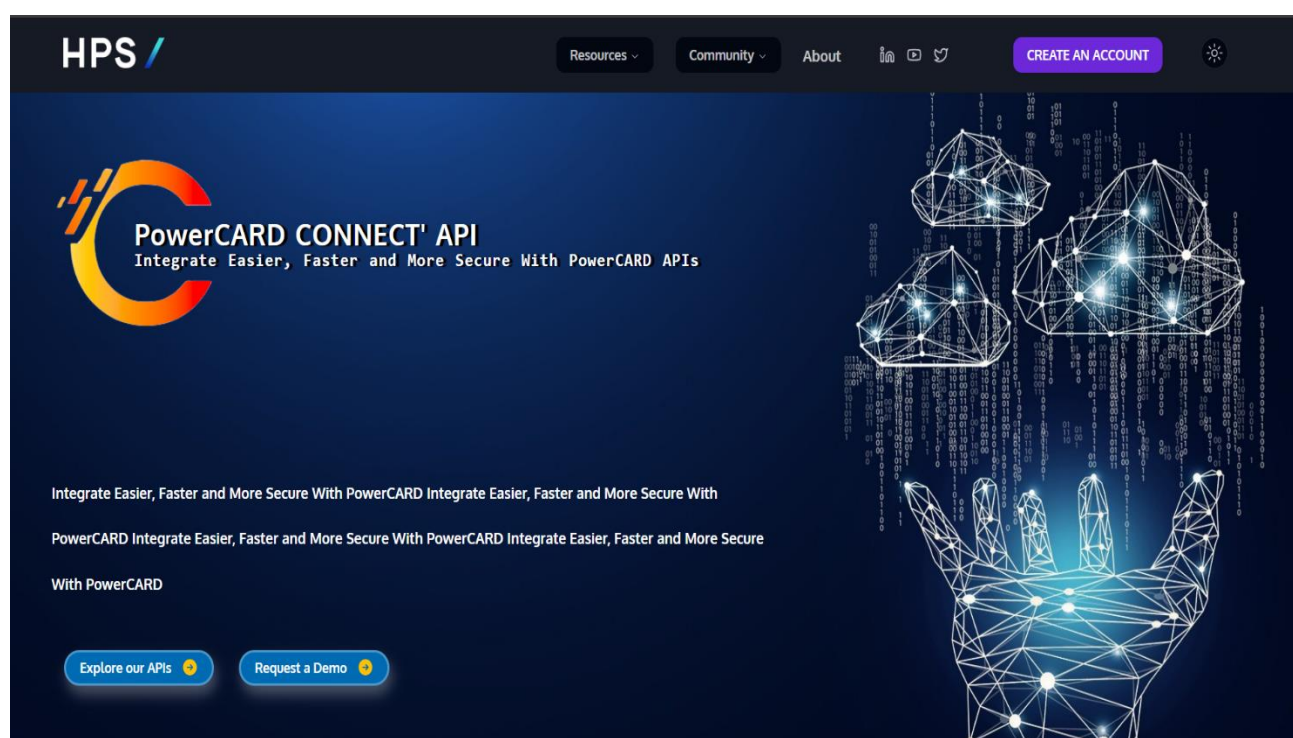


Figure 33 - L'interface Accueil sandbox-ui

5.1.2 L'interface Login

La figure 34 ci-dessous, est la page de login. L'utilisateur saisi l'email ou nom d'utilisateur et le mot de passe. Quand l'utilisateur lance l'application web, la page d'authentification s'affiche. L'utilisateur doit saisir ces informations d'authentification.

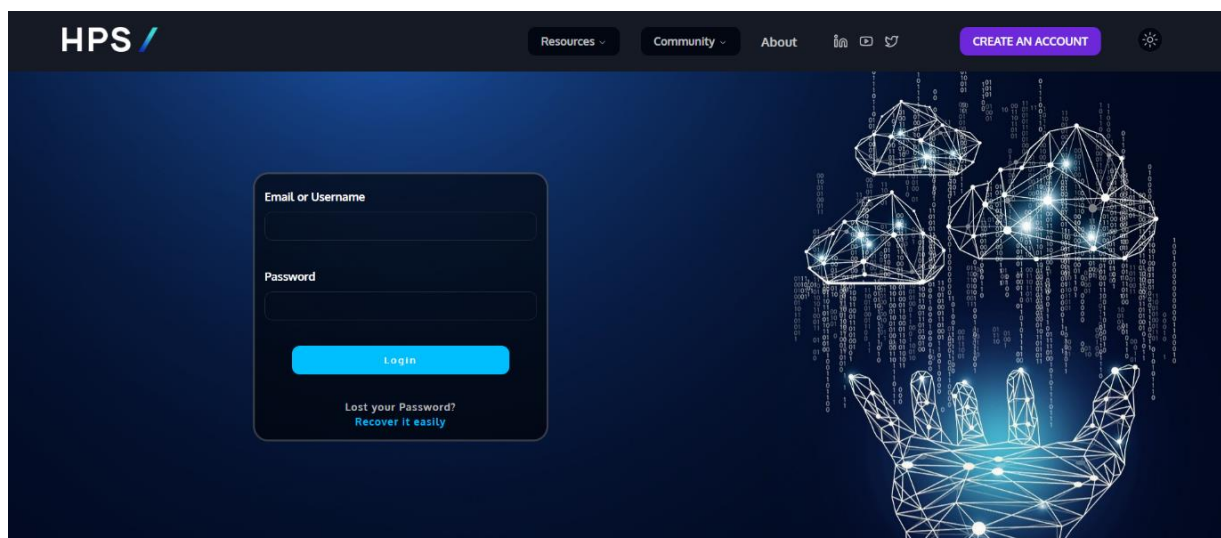


Figure 34 - l'interface Login sandbox-ui

5.1.3 L'interface inscription

La figure 35 ci-dessous illustre la page d'inscription. L'utilisateur y entre son nom complet, son nom d'utilisateur, son adresse email, son numéro de téléphone et son mot de passe. Si les informations saisies sont conformes, le formulaire est envoyé au serveur pour vérification. Le serveur renvoie ensuite une réponse à l'application indiquant si l'inscription a été réussie. Une demande d'inscription est alors envoyée à l'administrateur, qui décide d'accepter ou non la demande.

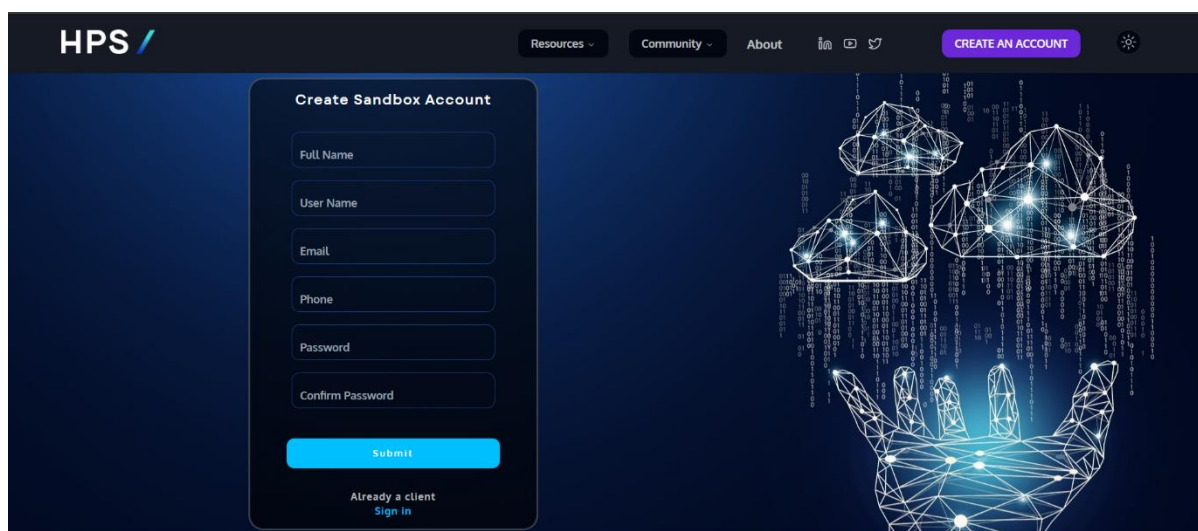


Figure 35 - L'interface inscription sandbox-ui

5.1.4 L'interface produits

Les figures 36 et 37 ci-dessous illustrent la page des produits. L'utilisateur choisi un produit, un produit représente une version de la solution PowerCARD.

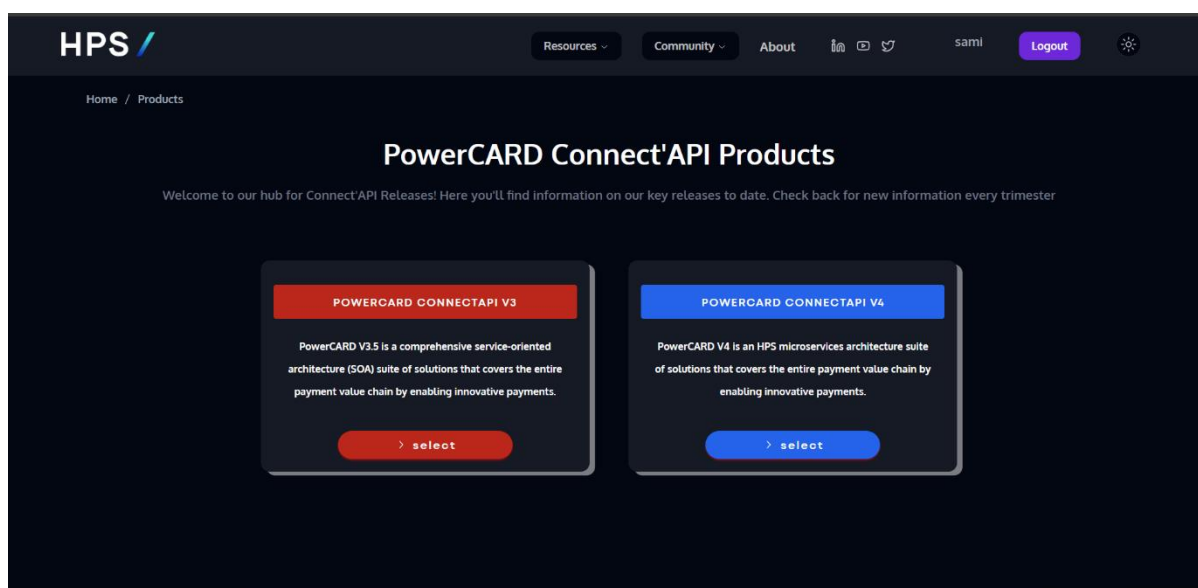


Figure 36 - L'interface Produits dark mode sandbox-ui

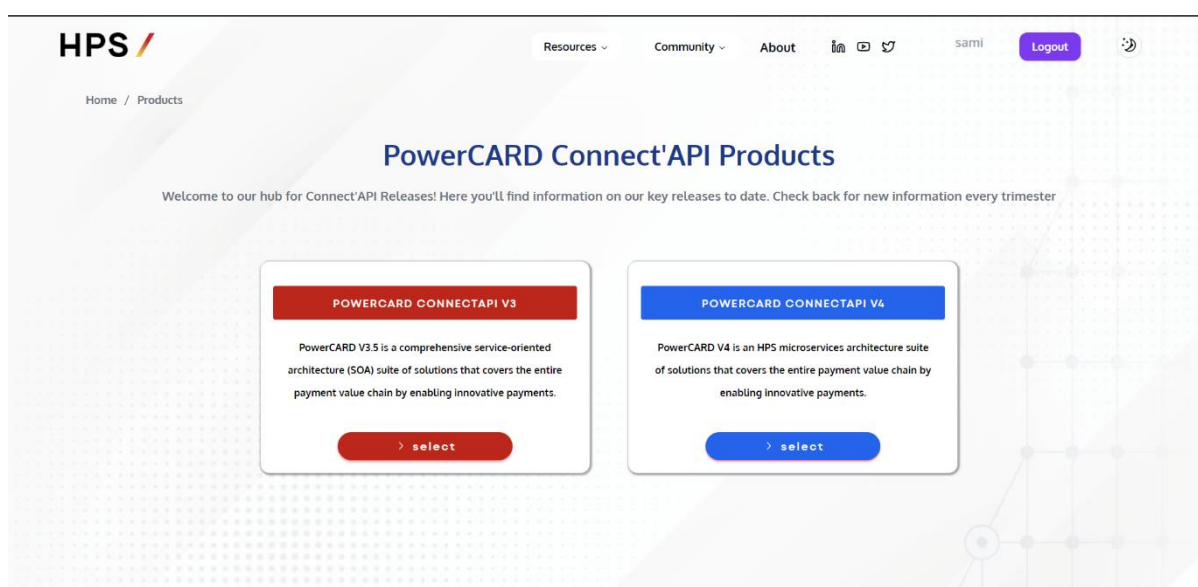


Figure 36 - L'interface Produits light mode Sandbox-ui

5.1.5 L'interface product release

La figure 38 ci-dessous illustre la page des product releases. Où nous trouverons des informations sur les versions réalisées jusqu'à présent.

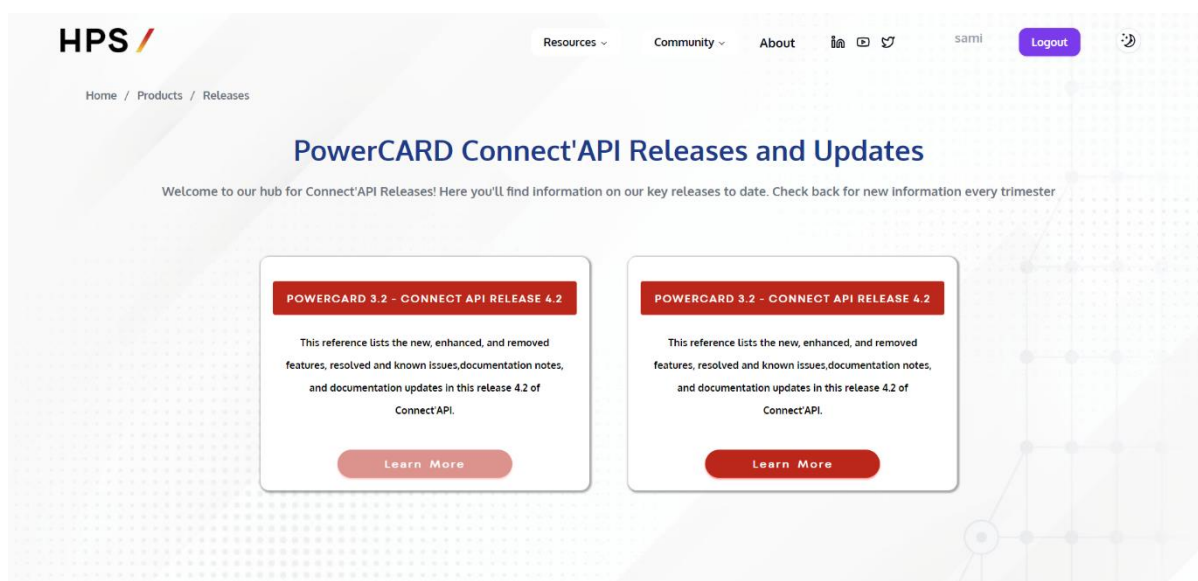


Figure 37 - L'interface product release sandbox-ui

5.1.6 L'interface Solution

La figure 39 ci-dessous illustre la page des solutions fournies dans PowerCARD. L'utilisateur peut explorer chaque solution en cliquant sur Explore.

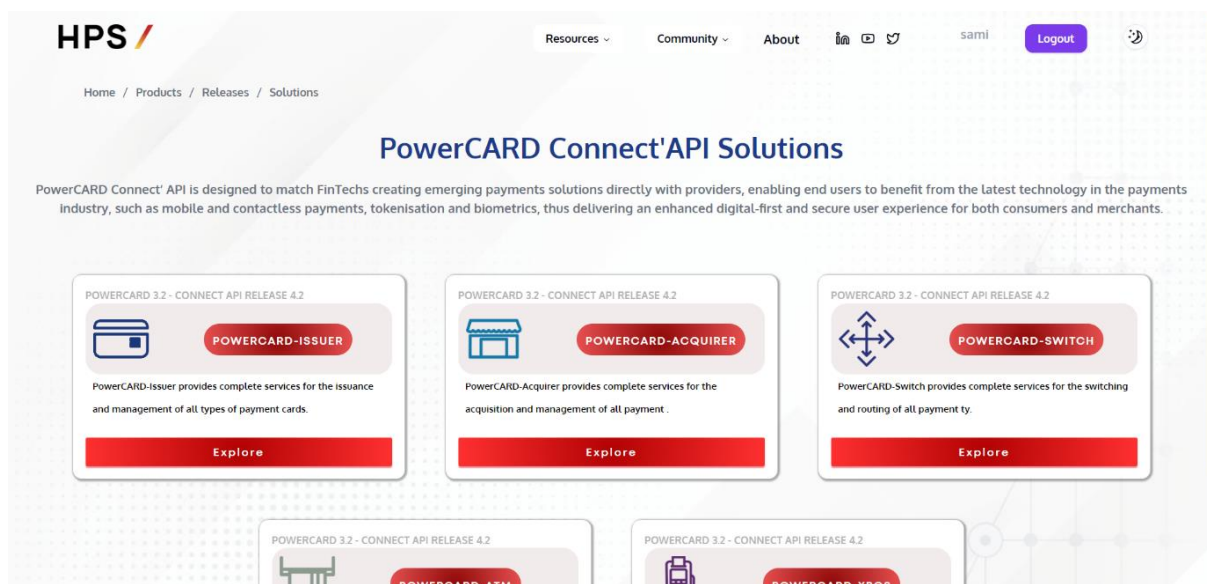


Figure 38 - L'interface Solution sandbox-ui

5.1.7 L'interface Module

La figure 40 ci-dessous illustre la page des Modules de chaque solution présentée dans la figure 39.

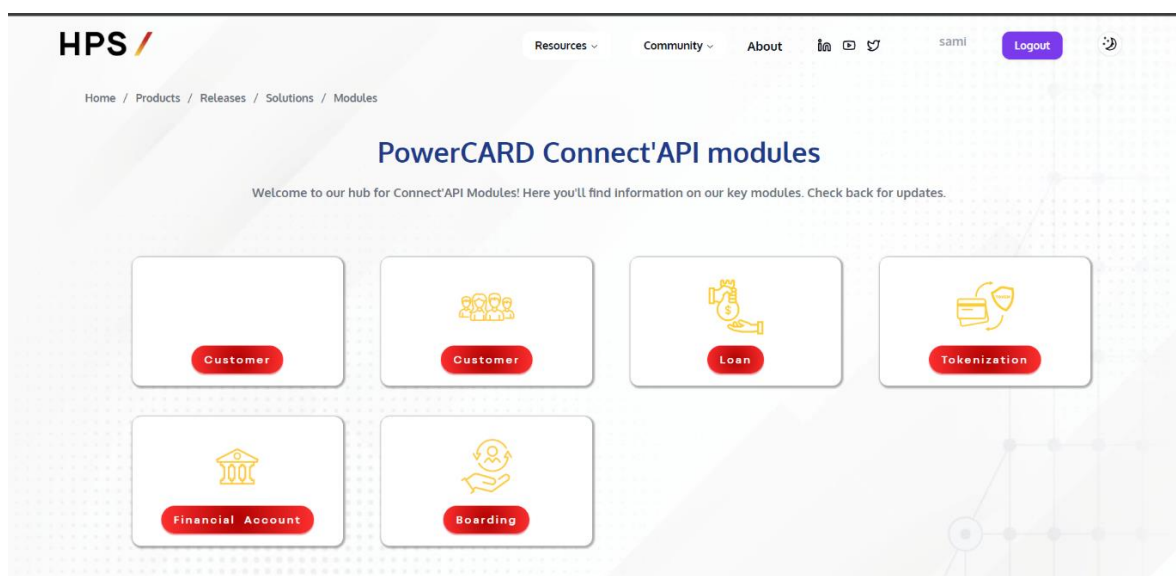


Figure 39 - L'interface module sandbox-ui

5.1.8 L'interface API

La figure 41 ci-dessous illustre la page de documentation des API. L'utilisateur peut y explorer les APIs répertoriées dans le menu " API Overview" situé à gauche de la page. Il peut également accéder à la description détaillée de chaque API.

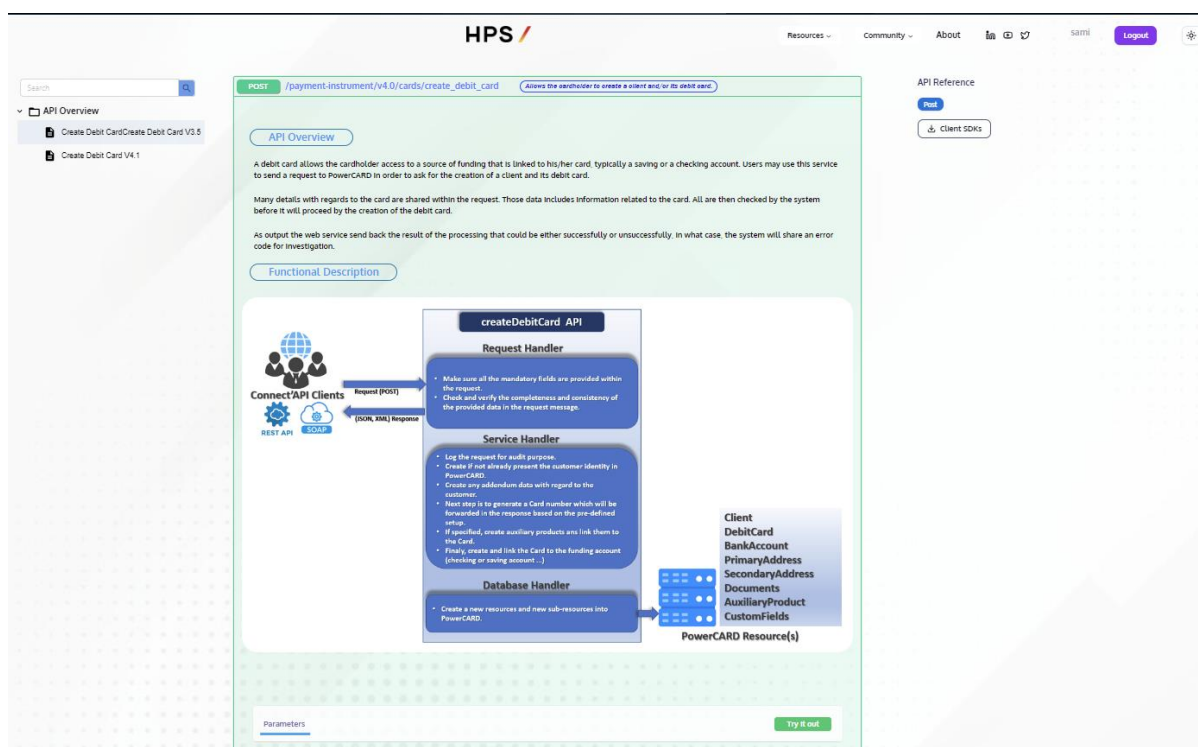


Figure 40 - L'interface Api sandbox-ui

5.2 Sandbox-Admin

La Sandbox-Admin est dédiée aux administrateurs, offrant un panel complet pour la gestion des APIs, des utilisateurs, des produits, des produits releases et modules. Elle permet aux administrateurs d'ajouter de nouvelles APIs, de gérer les comptes utilisateurs et gérer tous les composants de l'application.

5.2.1 L'interface produits

La figure 42 ci-dessous présente la page de gestion des produits par l'administrateur. Cette page permet à l'administrateur de gérer les produits PowerCARD en définissant leur visibilité pour les utilisateurs et en activant ou désactivant la possibilité d'y accéder.

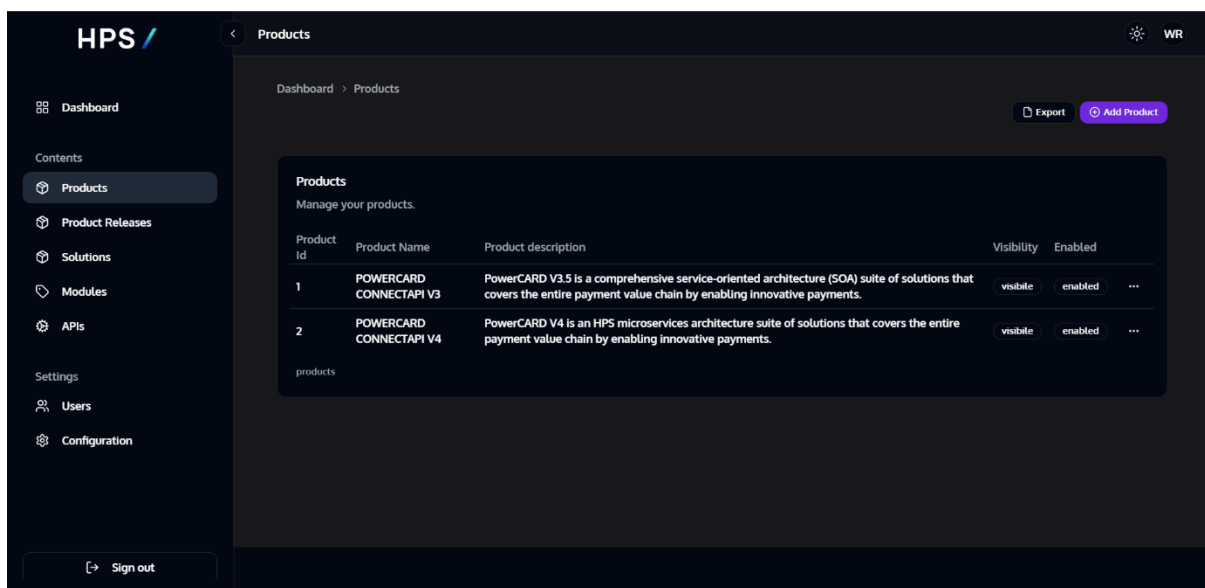


Figure 41 - L'interface produit sandbox-admin

5.2.2 L'interface product release

La figure 43 ci-dessous présente la page de gestion des products releases par l'administrateur. Cette page permet à l'administrateur de gérer les products releases PowerCARD en définissant leur visibilité pour les utilisateurs et en activant ou désactivant la possibilité d'y accéder.

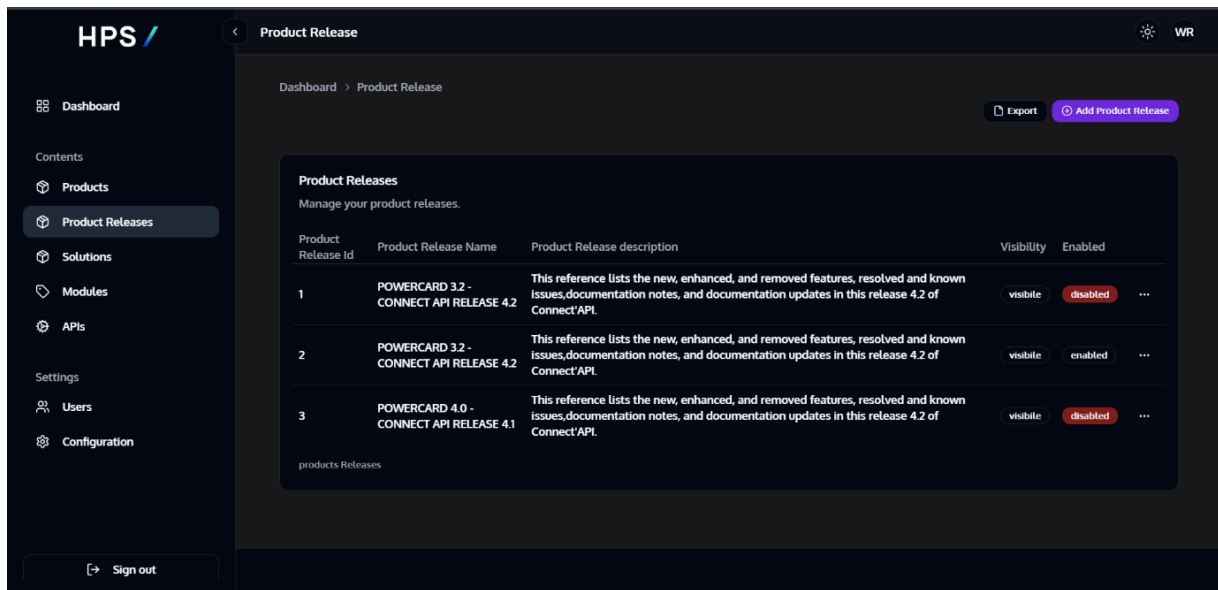


Figure 42 - l'interface product release sandbox-admin

5.2.3 L'interface Solution

La figure 44 ci-dessous présente la page de gestion des solutions par l'administrateur. Cette page permet à l'administrateur de gérer les solutions PowerCARD en définissant leur visibilité pour les utilisateurs et en activant ou désactivant la possibilité d'y accéder.

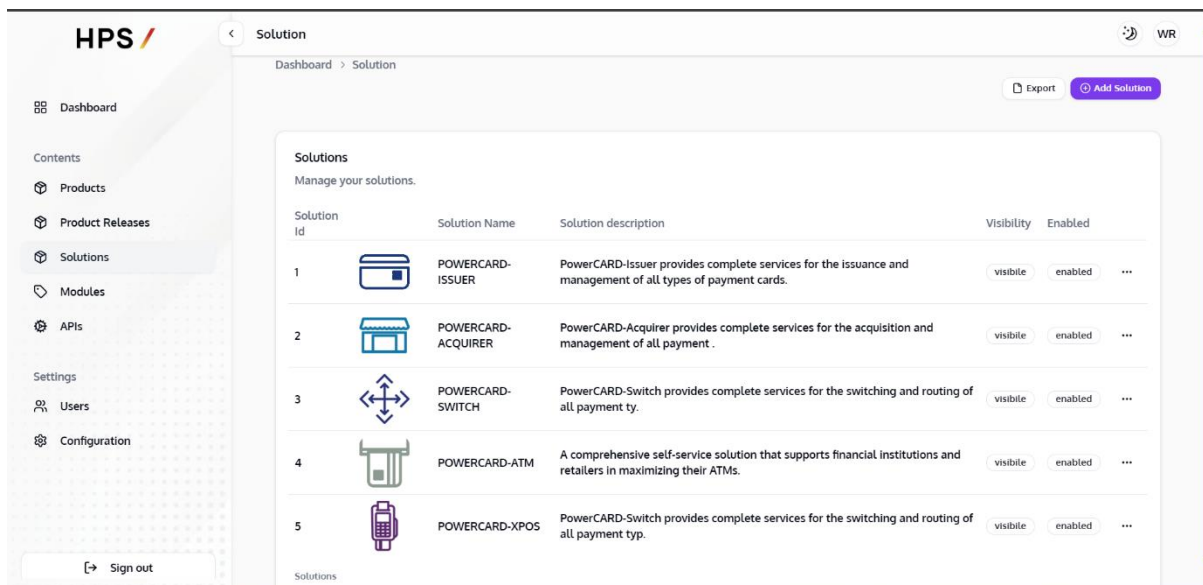


Figure 43 – L'interface solution sandbox-admin

5.2.4 L'interface Modules

La figure 45 ci-dessous présente la page de gestion des modules accessible à l'administrateur. Cette page permet à l'administrateur de gérer les modules de PowerCARD en définissant leur visibilité pour les utilisateurs et en activant ou désactivant la possibilité d'y accéder.

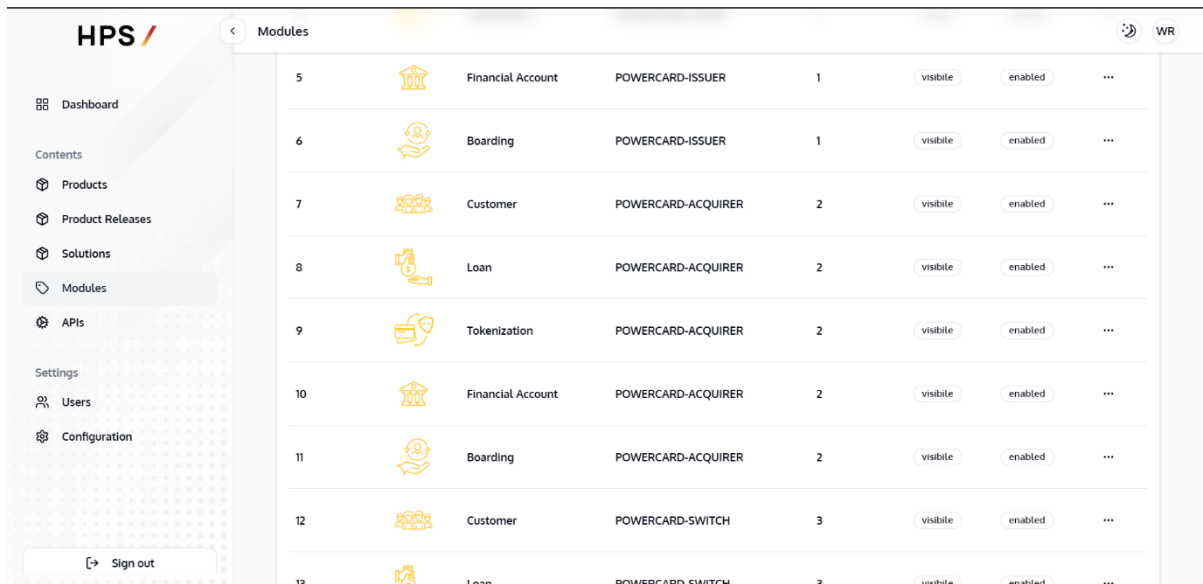


Figure 44 - L'interface module sandbox-admin

5.2.5 Gestion des utilisateurs

5.2.5.1 L'interface utilisateur

La figure 46 ci-dessous illustre la page de gestion des utilisateurs accessible à l'administrateur. Cette page permet à l'administrateur de gérer les utilisateurs de PowerCARD en consultant leurs informations et en effectuant des actions sur eux.

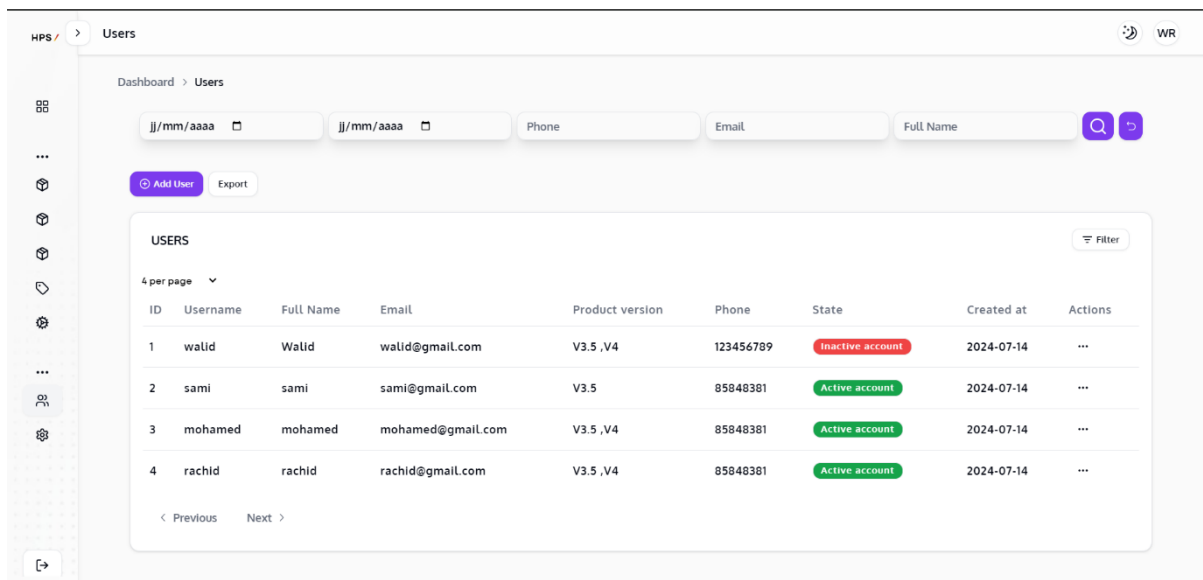


Figure 45 - L'interface utilisateur sandbox-admin

5.2.5.2 L'interface d'ajout d'un utilisateur

La figure 47 ci-dessous présente la page d'ajout d'un utilisateur. Cette page permet à l'administrateur de créer de nouveaux utilisateurs en saisissant leurs informations et en activant leurs comptes.

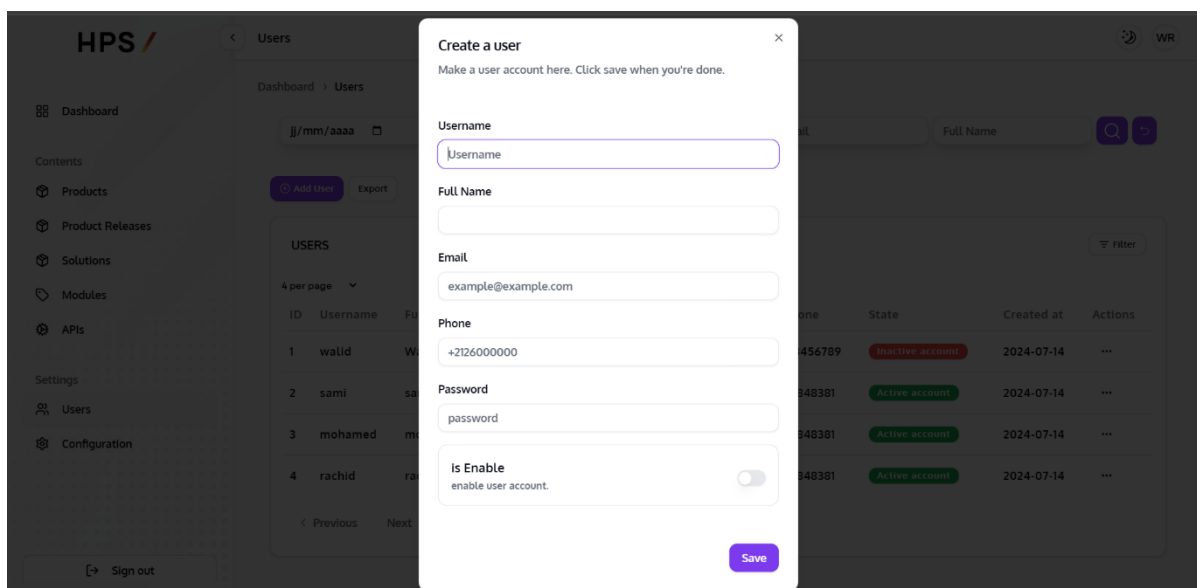


Figure 46 - L'interface d'ajout d'un utilisateur sandbox-admin

5.2.5.3 L'interface de modification d'un utilisateur

La figure 48 ci-dessous présente la page de modification d'un utilisateur. Cette page permet à l'administrateur de modifier les informations d'un utilisateur existant, y compris son nom, son adresse e-mail et son numéro de téléphone. L'administrateur peut également activer ou désactiver le compte de l'utilisateur.

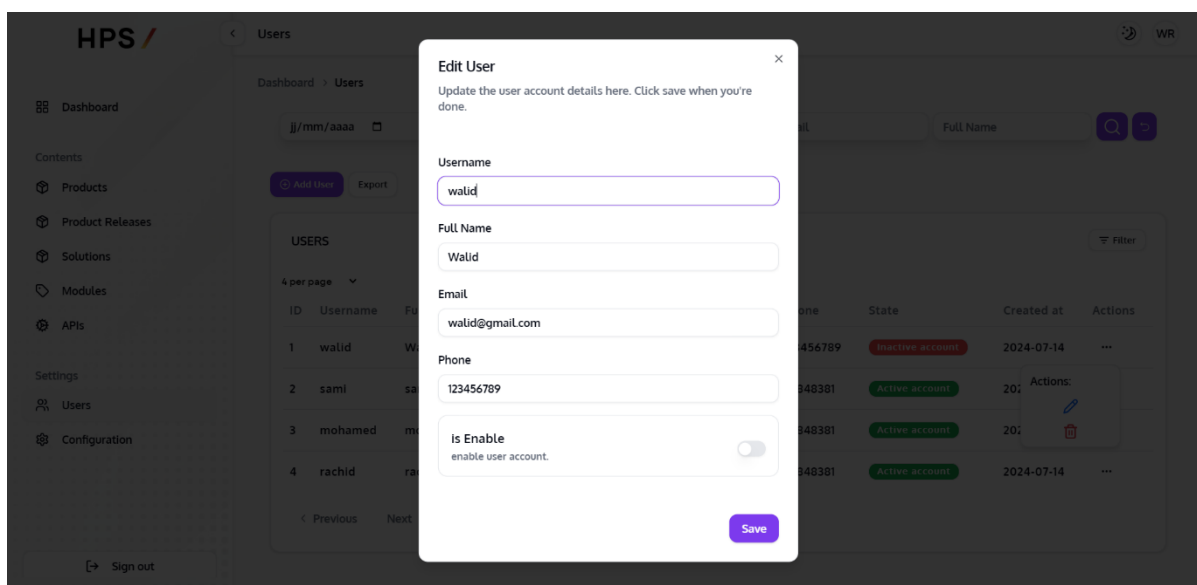


Figure 47 - L'interface de modification d'un utilisateur sandbox-admin

5.2.5.4 L'interface de suppression d'un utilisateur

La figure 49 ci-dessous présente la page de suppression d'un utilisateur. Cette page permet à l'administrateur de supprimer un utilisateur existant de la base de données PowerCARD. La suppression d'un utilisateur est définitive et ne peut pas être annulée.

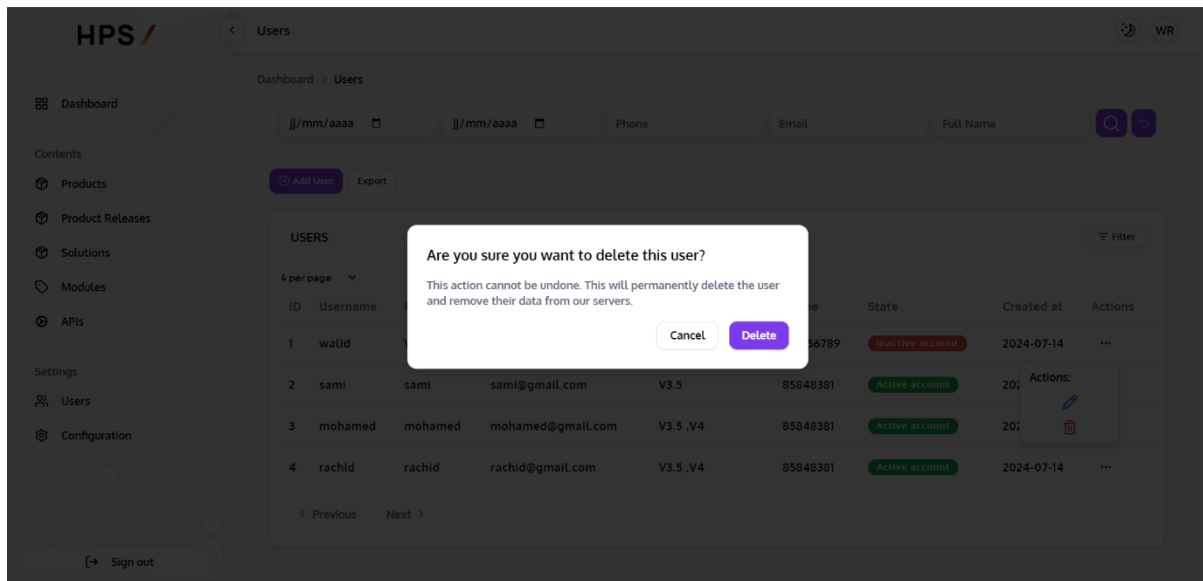


Figure 48 - L'interface de suppression d'un utilisateur sandbox-admin

Conclusion

En conclusion, ce chapitre a présenté les interfaces utilisateur de la solution PowerCARD ainsi que les résultats obtenus durant la phase de réalisation du projet. Des captures d'écran illustrant les différentes fonctionnalités de l'application sont également incluses pour offrir une meilleure compréhension de l'outil et de ses capacités.

Conclusion générale

En conclusion, ce projet de fin d'étude a permis de mettre en évidence l'importance cruciale des APIs dans le développement logiciel moderne. En se concentrant sur l'optimisation de la Sandbox ConnectAPI, nous avons réussi à créer un espace administratif intuitif, facilitant la gestion des documentations d'API et offrant une personnalisation avancée des espaces clients. Grâce à une interface graphique flexible et conviviale, les administrateurs peuvent désormais gérer les APIs de manière efficace et adapter l'interface aux besoins spécifiques de chaque client.

Ce rapport a été structuré en cinq chapitres pour offrir une vue complète du projet. Le contexte général a posé les bases en définissant le cadre, les objectifs et les enjeux du projet. Les spécifications et l'analyse ont permis d'identifier les limitations de l'environnement actuel et de reformuler précisément la problématique à résoudre. L'étude conceptuelle a présenté les solutions envisagées pour répondre à cette problématique, tandis que l'architecture technique a détaillé les technologies et la structure logicielle et matérielle mises en place. Le chapitre de mise en œuvre a illustré les étapes de développement et les défis relevés.

En évaluant les réalisations du projet par rapport aux objectifs initiaux, il apparaît clairement que les améliorations apportées ont significativement optimisé la gestion des APIs et renforcé l'interopérabilité et la modularité des systèmes logiciels. Pour l'avenir, les perspectives d'évolution incluent l'intégration de nouvelles fonctionnalités et l'amélioration continue de l'interface utilisateur pour répondre aux besoins en constante évolution des développeurs et des utilisateurs finaux. Ce projet souligne la dynamique d'innovation et de collaboration essentielle à l'ère des APIs, ouvrant la voie à de nouvelles opportunités dans le domaine du développement logiciel.

Webographie

Le site hps des apis : <https://sandbox.connectapi.hps-worldwide.com/>

shadcn : <https://ui.shadcn.com/>

flyway : <https://flywaydb.org/>

open api docs : <https://swagger.io/docs/specification/about/>

tailwindcss docs : <https://v2.tailwindcss.com/docs>